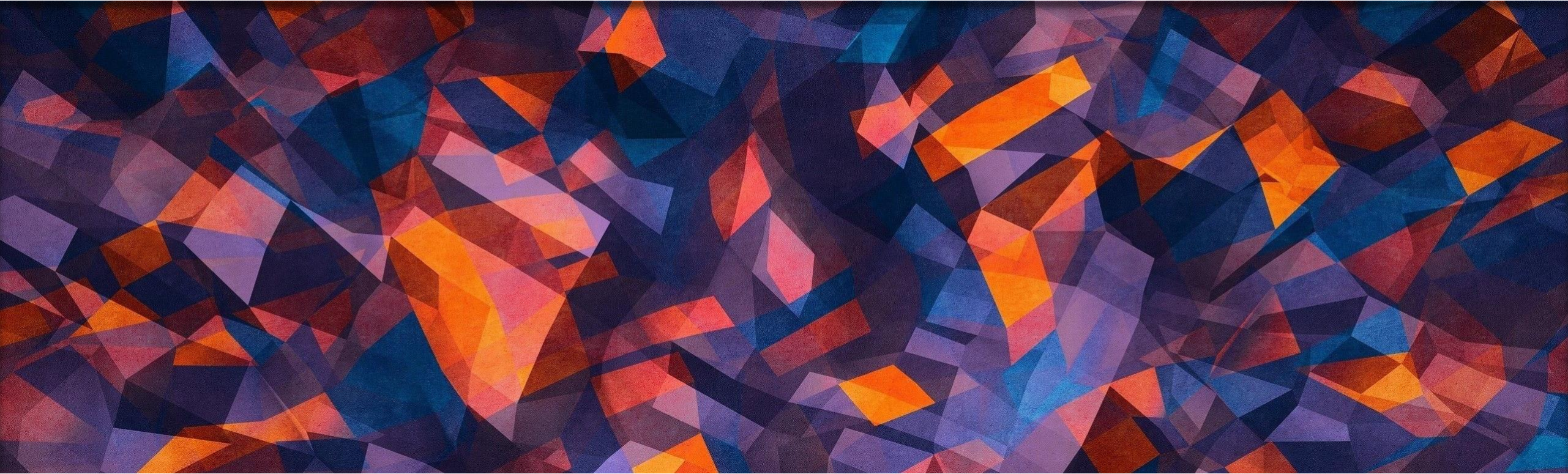




Universität St.Gallen



Event-driven and Process-oriented Architectures

Stream Processing Design Patterns

Lecture 8

Event Processing Design Patterns

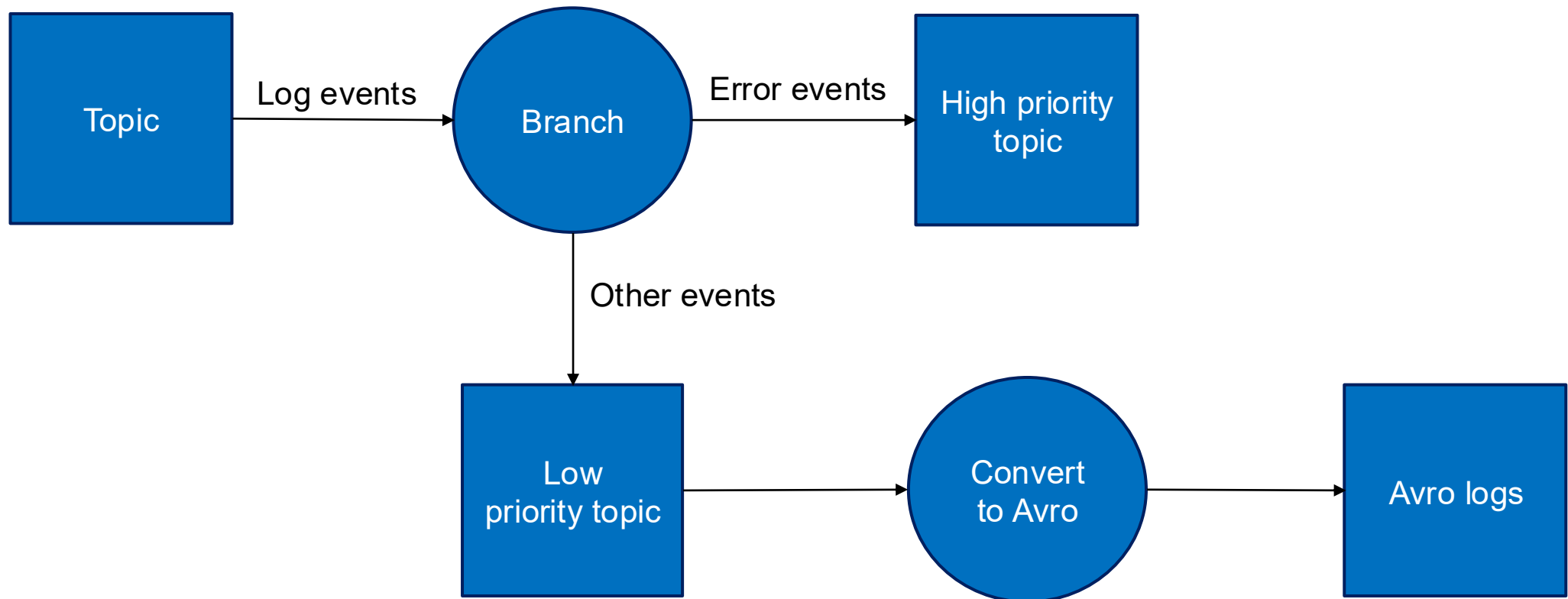
- Single-Event Processing
- Processing with Local State
- Multiphase Processing / Repartitioning
- Processing with External Lookup: Stream-Table join
- Table-Table Join
- Streaming Join
- Out-of-Sequence Events
- Reprocessing
- Interactive Queries

Section based on Kafka - The Definite Guide

Event Processing Design Patterns

Single-Event Processing

Most basic pattern of stream processing is the processing of each event in isolation (also known as *map/filter* pattern)

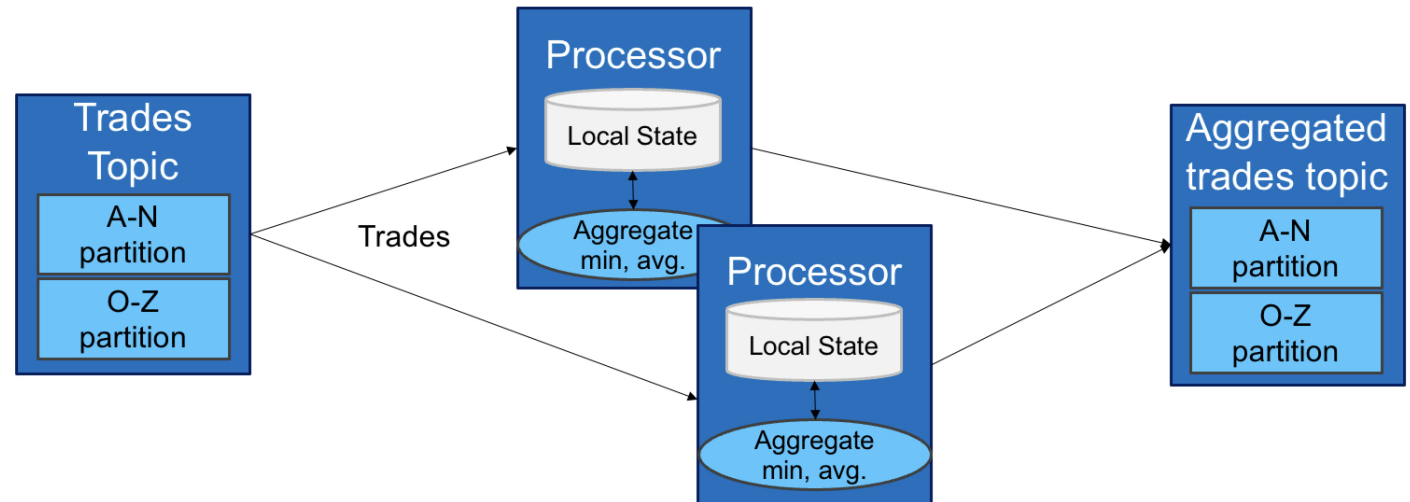


Event Processing Design Patterns

Processing with Local State

- Aggregations require maintaining state
- Events with the same key must be routed to the same partition
- Each process maintains local state

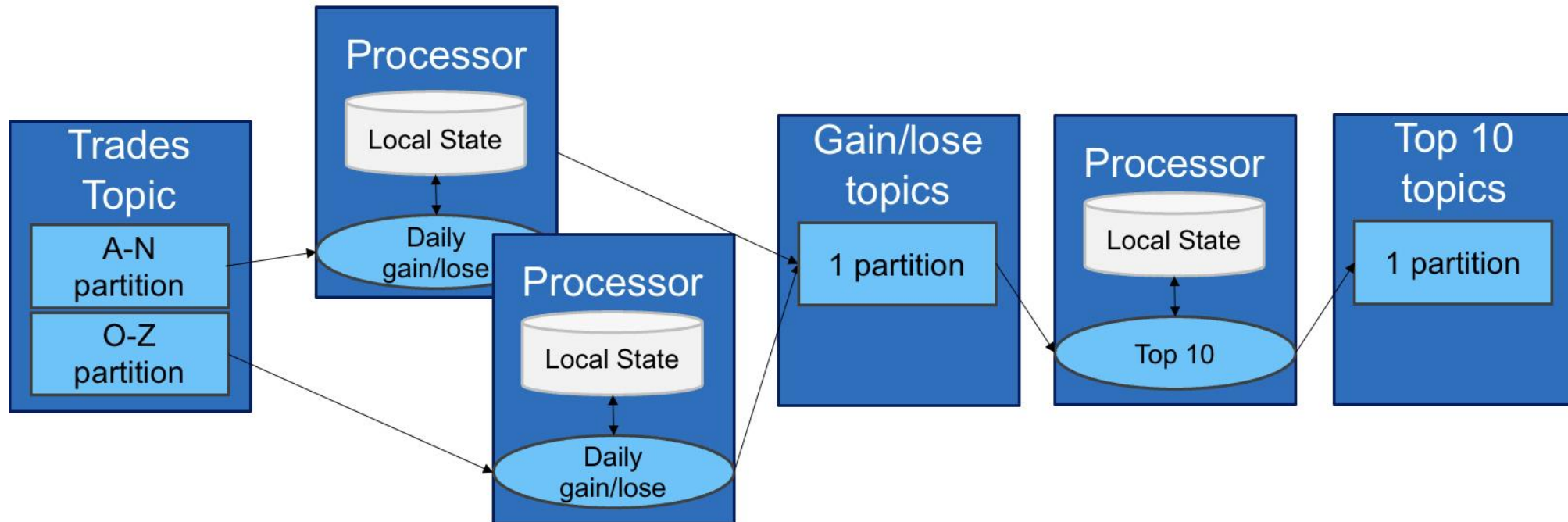
- Challenges:
 - Memory usage
 - Persistence
 - Rebalancing



Event Processing Design Patterns

Multiphase Processing / Repartitioning

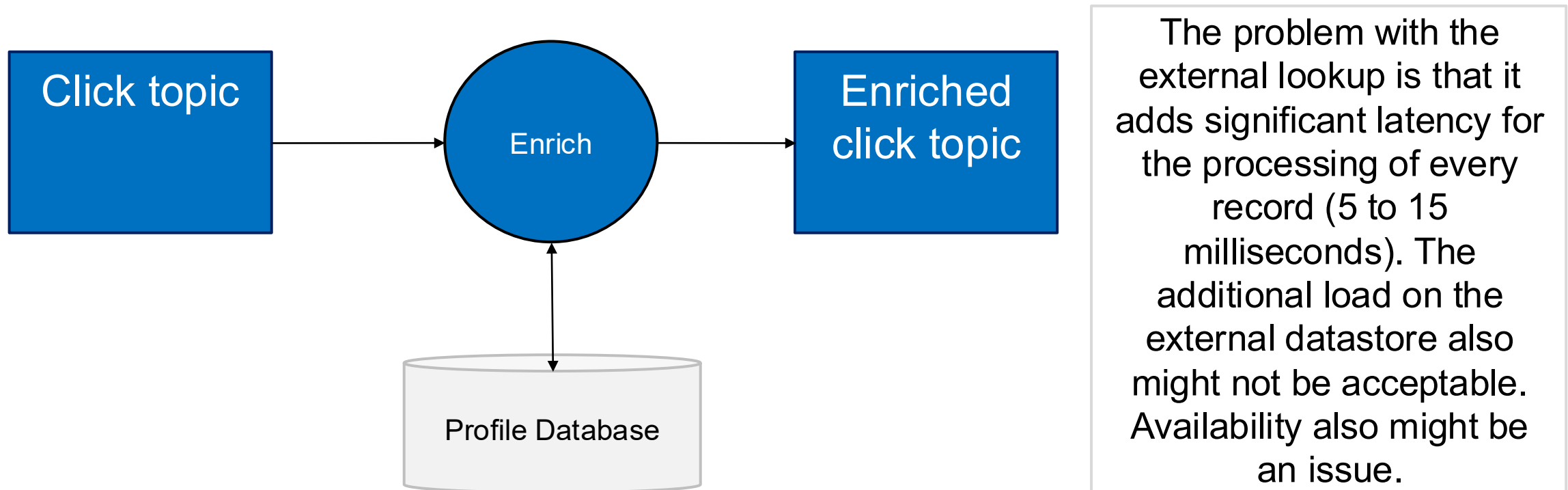
- First aggregate locally
- Repartition data
- Perform second aggregation



Event Processing Design Patterns

Processing with External Lookup: Stream-Table Join

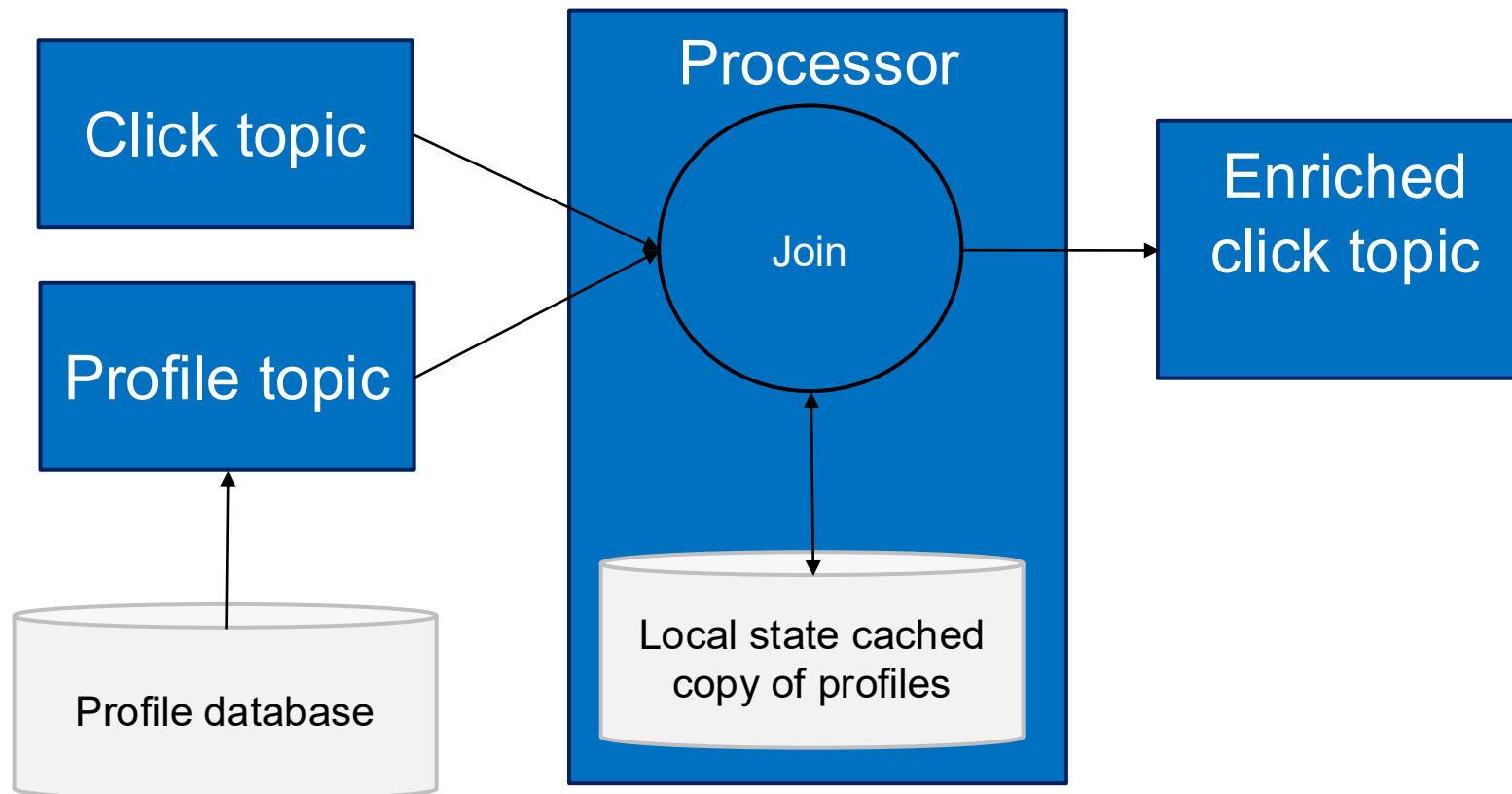
Sometimes stream processing requires integration with data external to the stream.



Event Processing Design Patterns

Processing with External Lookup: Stream-Table Join

Sometimes stream processing requires integration with data external to the stream.



To get good performance and availability, the information from the database needs to be cached in our stream processing application.

Idea: Capturing changes in database table in a stream of events; local database is updated upon changes

Event Processing Design Patterns

Table-Table Join

- Join two materialized tables
- Nonwindowed operation
- Uses latest state from both tables

Types:

- `equi-join` (same key)
- `foreign-key join` (lookup by different key)

Further details: <https://www.confluent.io/blog/data-enrichment-with-kafka-streams-foreign-key-joins/>

Event Processing Design Patterns

Streaming Join

When we join two streams, we join the entire history, try to match events in one stream with events in the other stream that have the same key and happened in the same time window (*windowed join*).

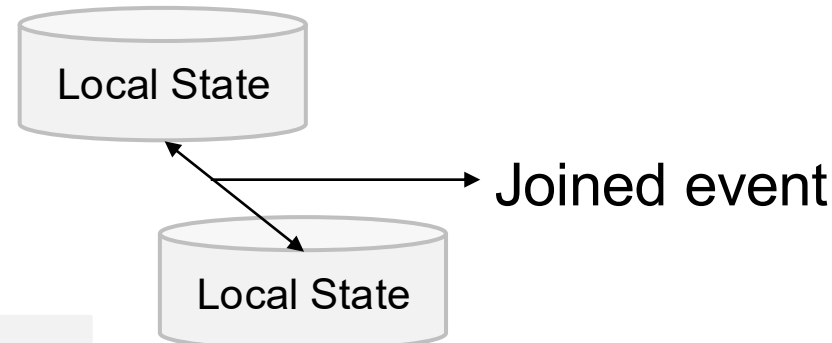
Clicks



Five-second window

Five-second window

Searches



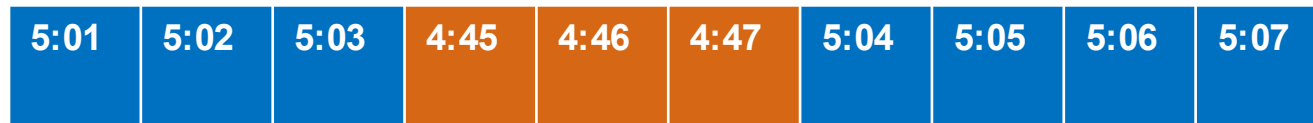
Event Processing Design Patterns

Out-of-Sequence Events

Events may arrive late

Processing must handle reordering

Grace period allows late arrivals



Older events arriving late

Detect out-of-order events and reconcile them during a grace period.

Event Processing Design Patterns

Reprocessing

Reprocessing

- Rerun application on existing event stream

Reasons

- Improved logic
- Bug fixes

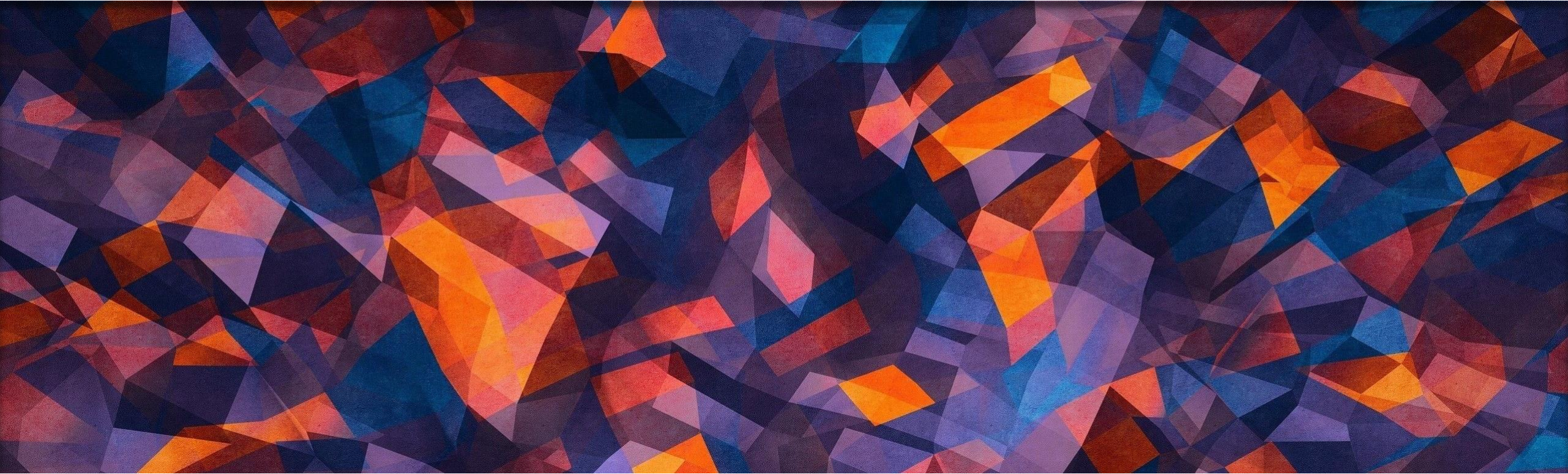
Options

- Run new version in parallel
- Reset offsets and recompute

Event Processing Design Patterns

Interactive Queries

- Read results from application state
- Avoid consuming output topics
- Enables low latency lookups

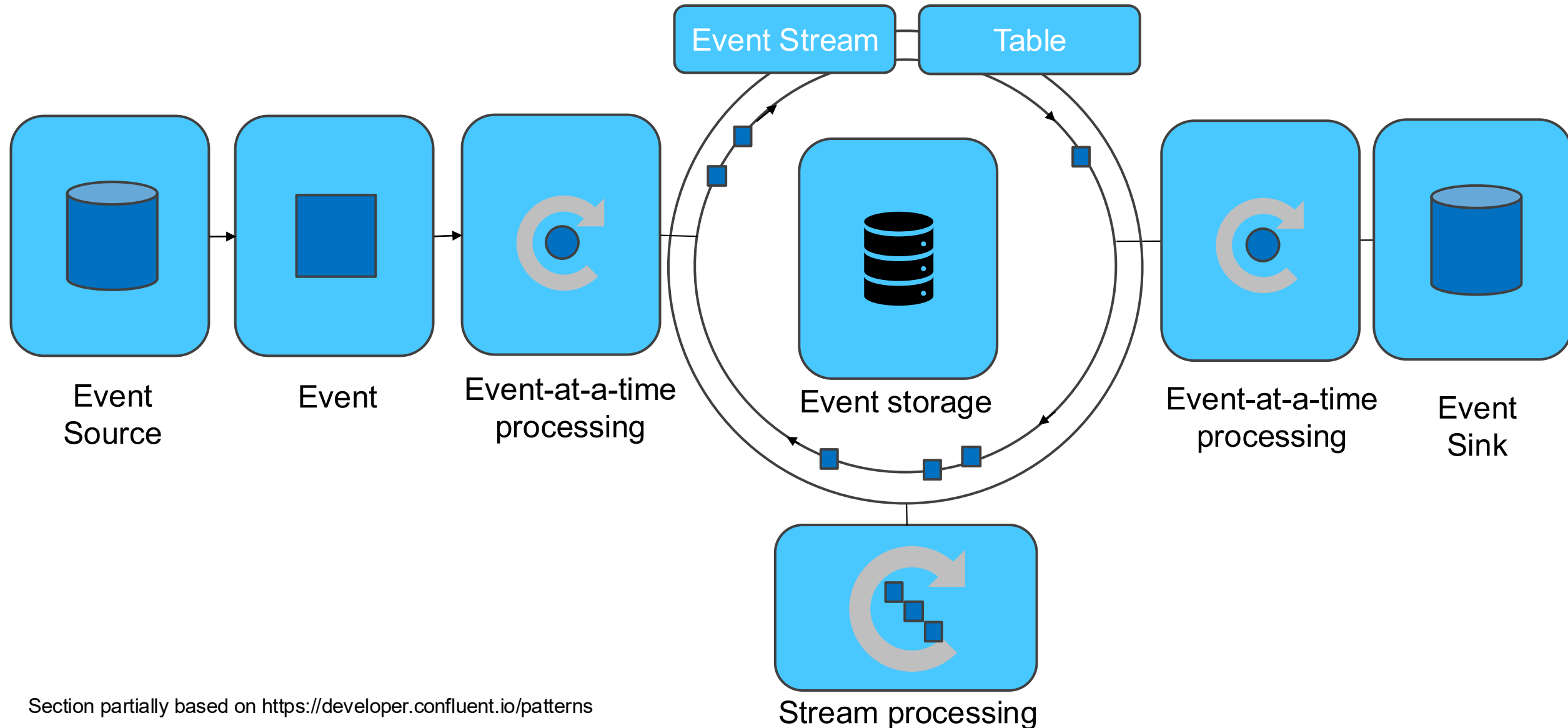


Event-driven and Process-oriented Architectures

Event Processing

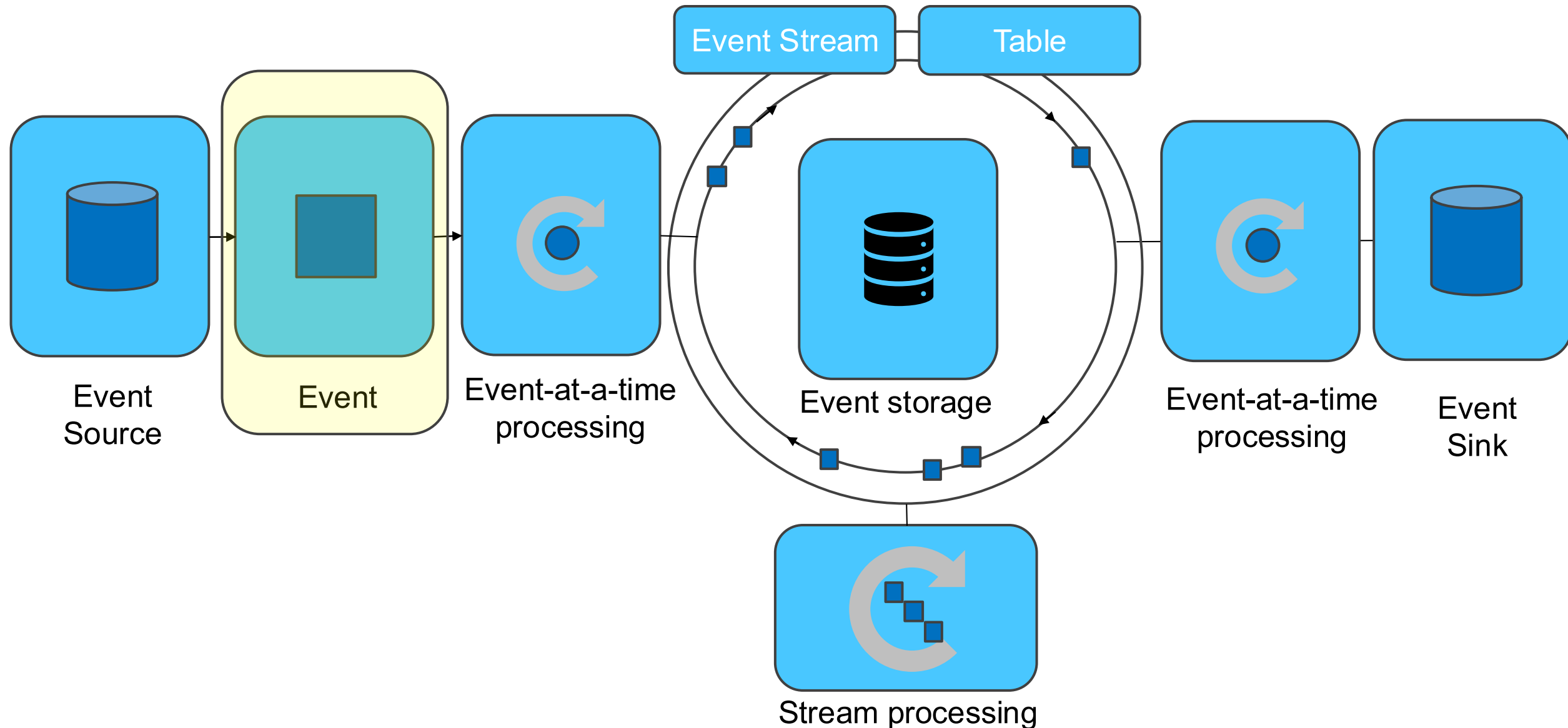
Lecture 8

Event Streaming Platform

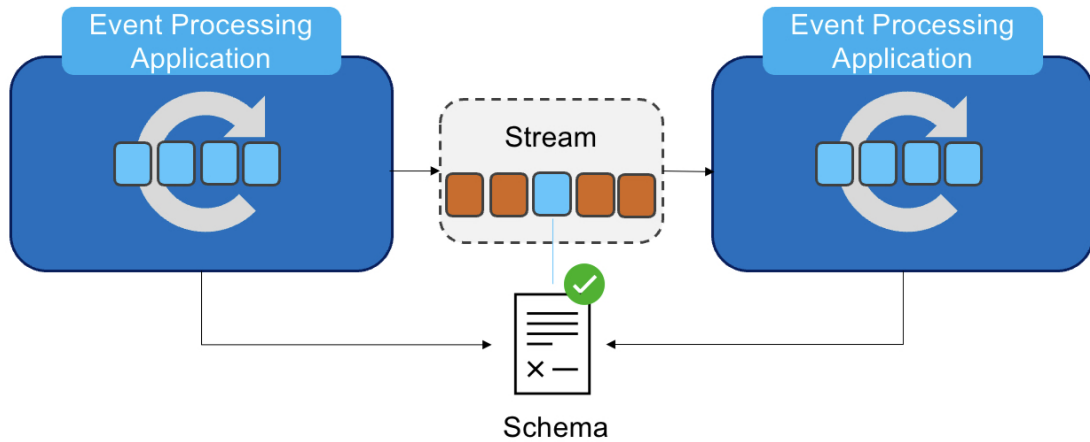


Section partially based on <https://developer.confluent.io/patterns>

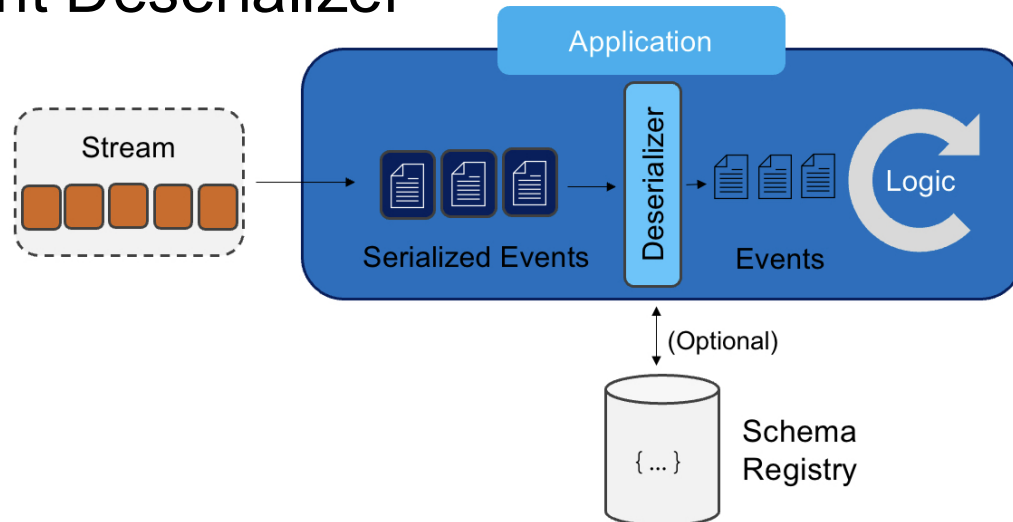
Event Streaming Platform



Data Contract

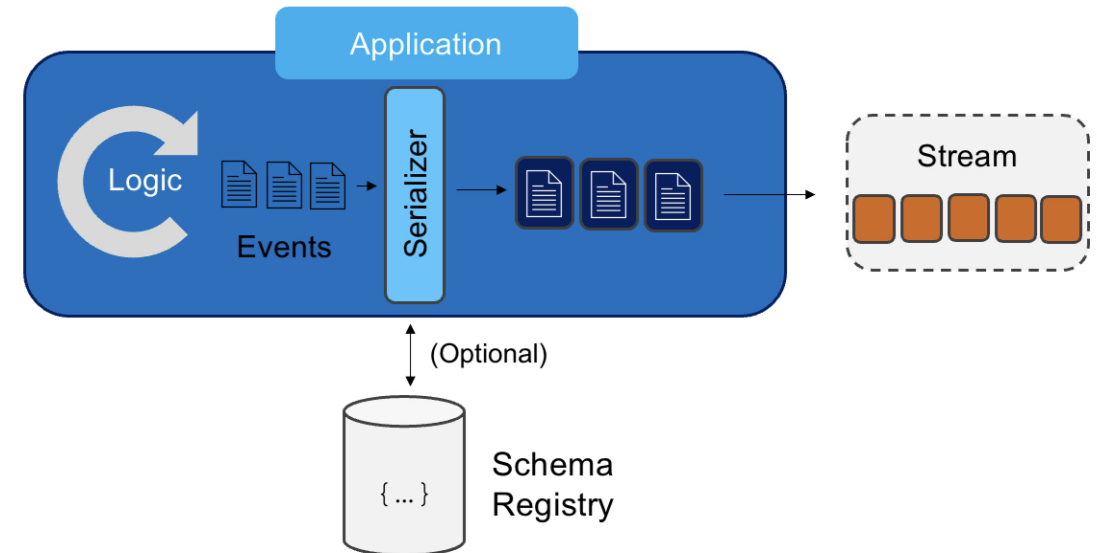


Event Deserializer

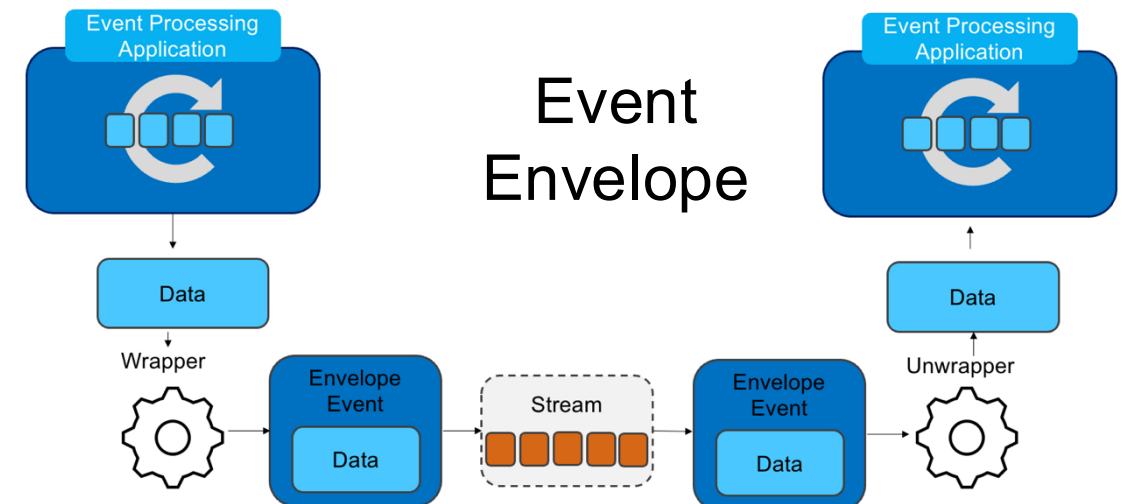


Selected Event Patterns

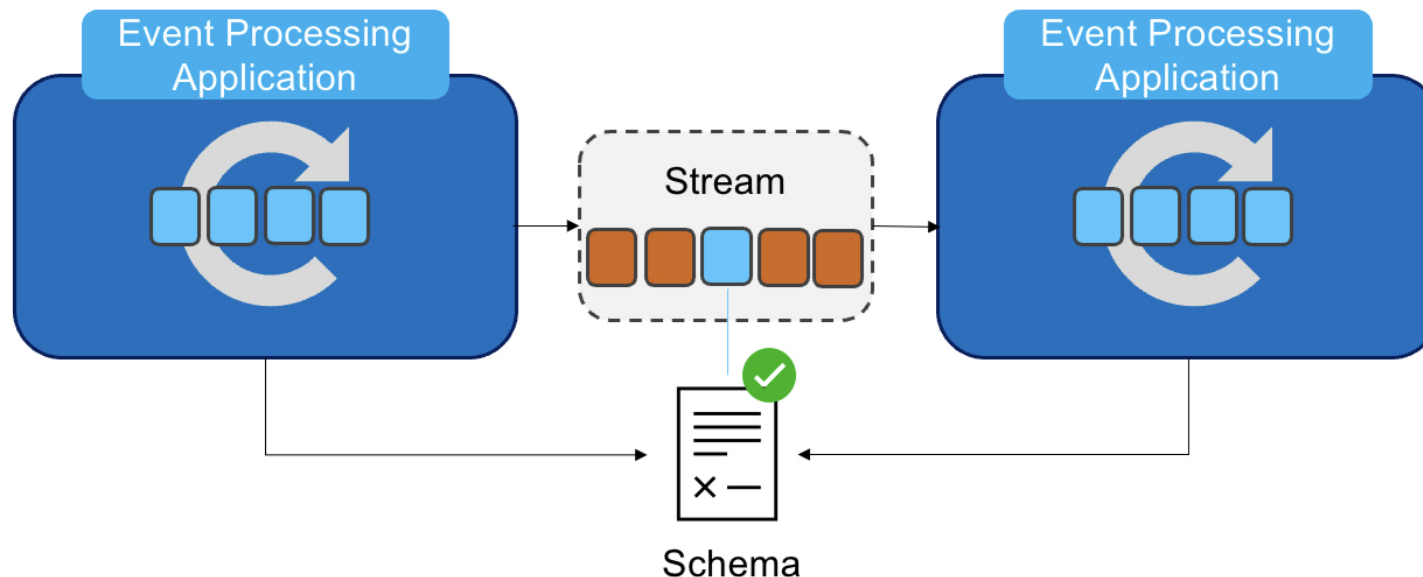
Event Serializer



Event Envelope

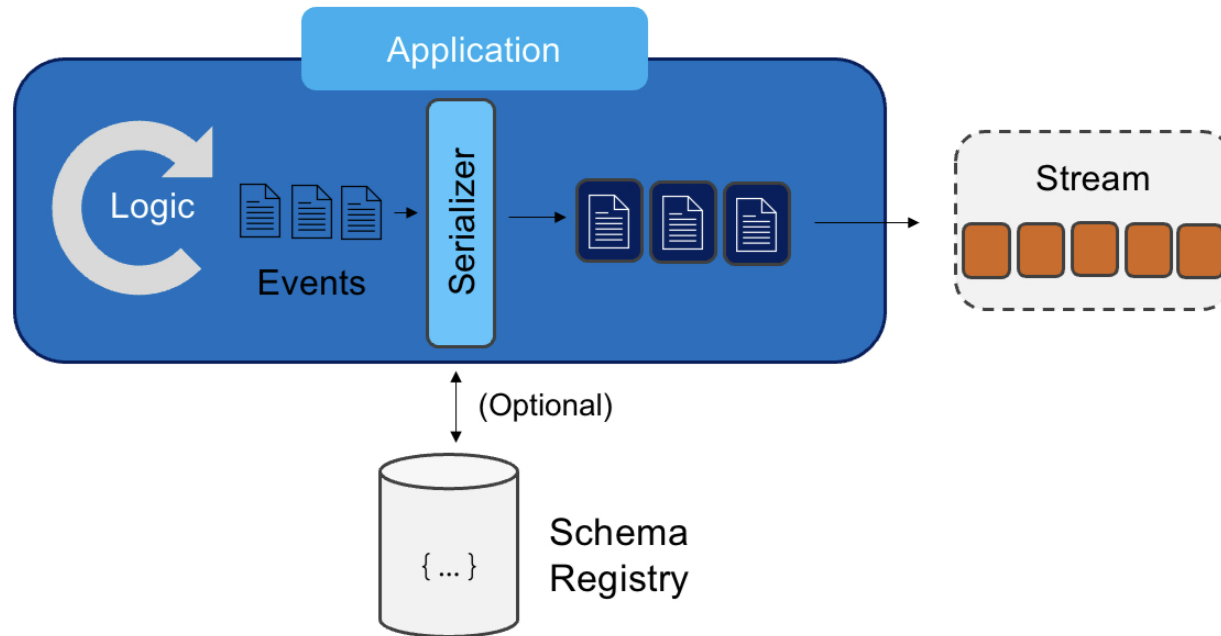


- When an Event Processing Application (EPA) sends an event to another EPA it is crucial that it understands how to process these shared events
- Using a data contract or a schema, EPAs can share events and understand how to process them, without the sending and the receiving application knowing any details about each other



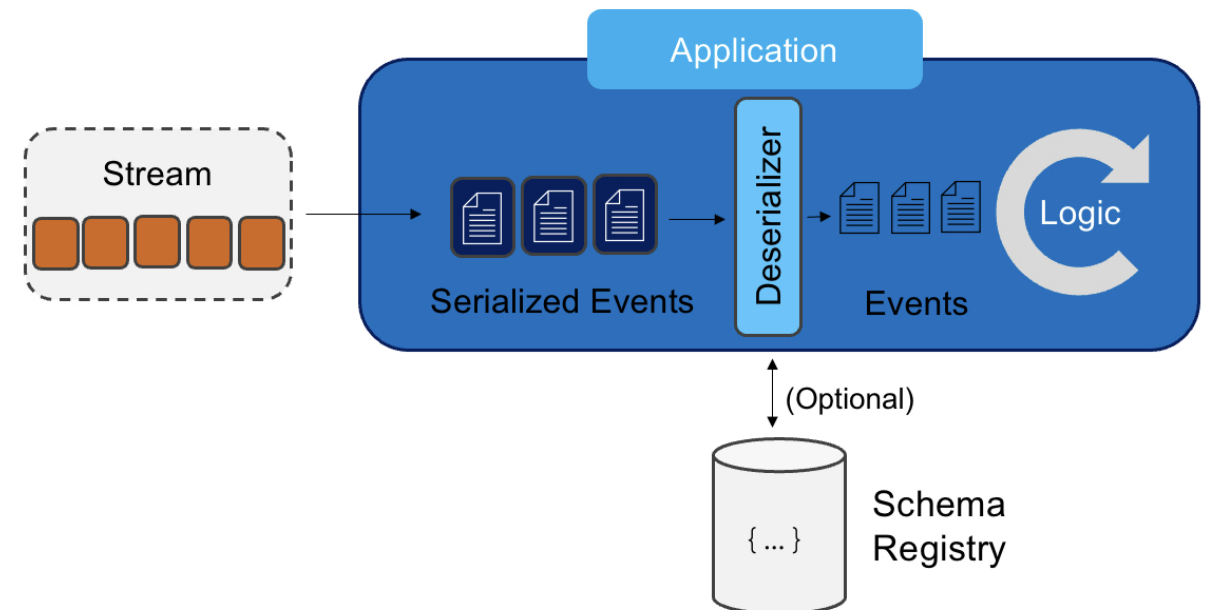
Event Serializer

- Convert events to binary format
- Use schema-based format (e.g., Avro)
- Optional: Use of schema registry



Event Deserializer

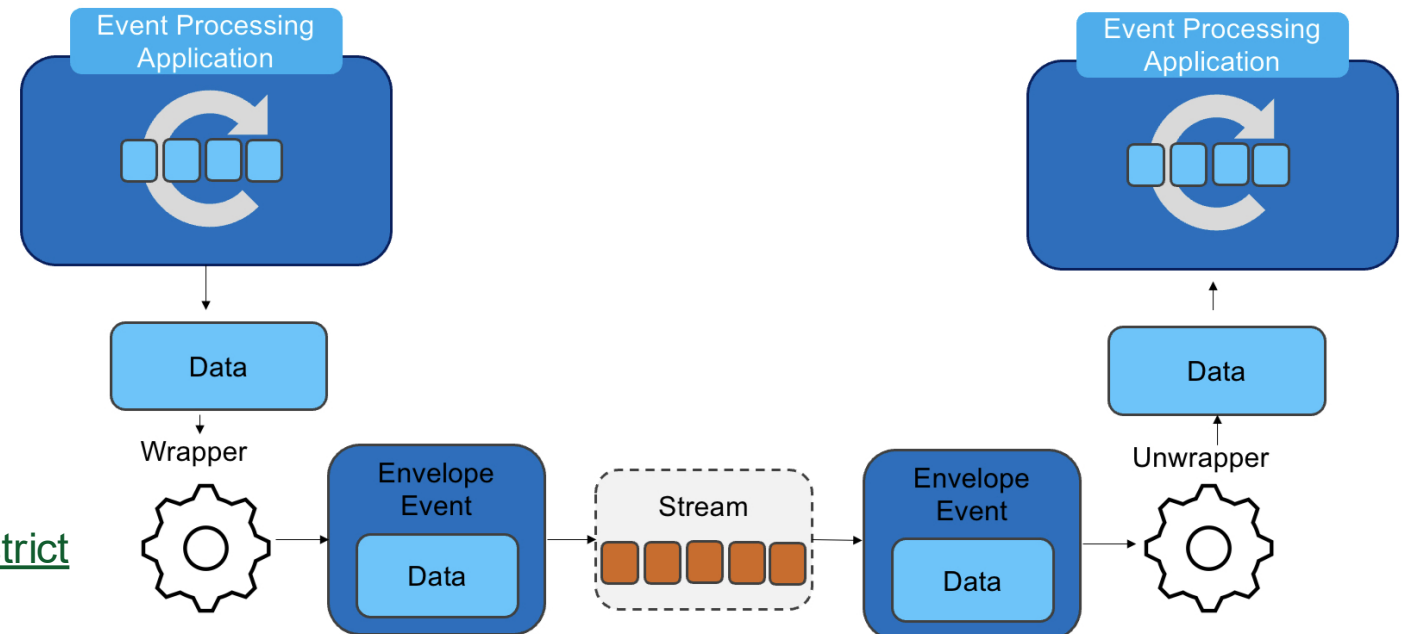
- To reconstruct the original event from the Event Streaming Platform use an Event Deserializer
- To make deserialization easy both the event data and its schema should be readily available; an avro library can use the schema to deserialize the events
- Optional: Use of schema registry



For more details: <https://developer.confluent.io/patterns/event/event-deserializer>

Event Envelope

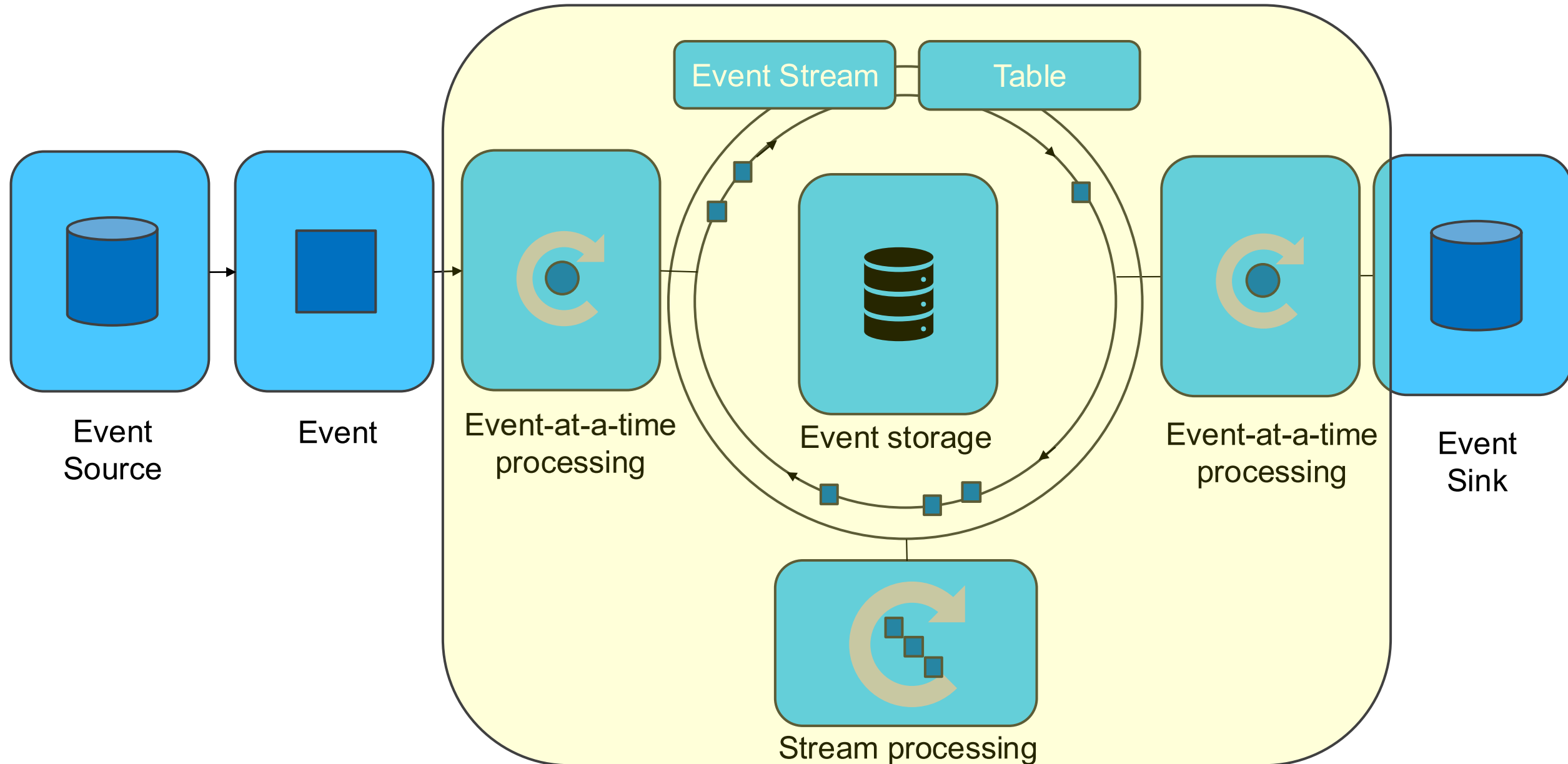
- An event envelope provides a standard set of fields across all events sent through the event processing applications. The envelope is independent of the underlying event format (analogous to protocol headers in networking)
- Cloud Events, which standardize access to ID, schema, key and other common event attributes, are an industry-standard example of the event envelope pattern



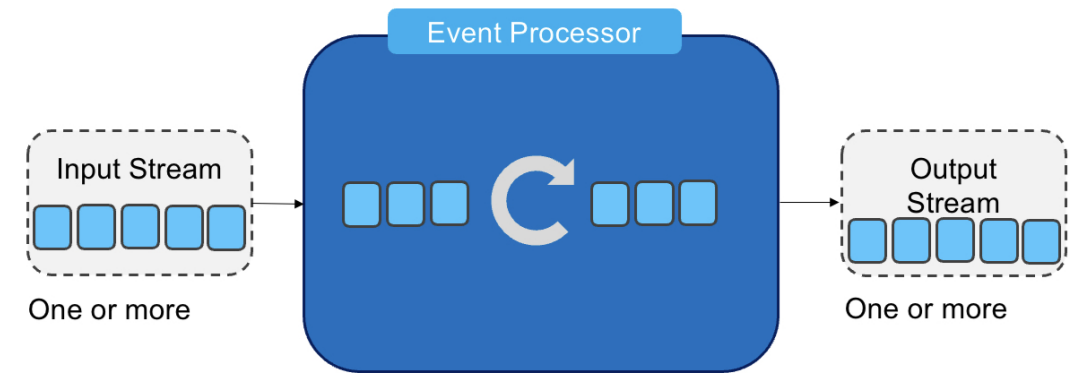
Pattern related to Envelope Wrapper in *Enterprise Integration Patterns*, by Gregor Hohpe and Bobby Woolf.

Further reference: [how to choose between strict and dynamic schemas](#)

Event Streaming Platform

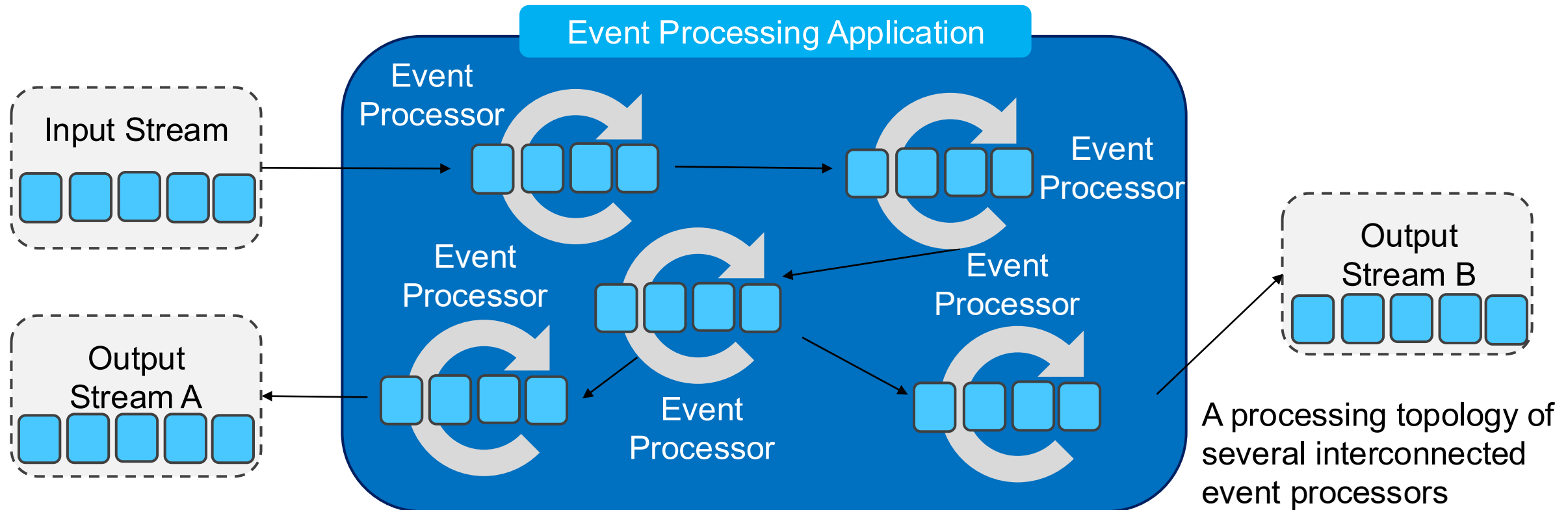


- Event Processors are components that
 - Read events,
 - process events,
 - write new events to output streams
- Event Processors
 - May act as an **Event Source** and/or **Event Sink**
 - Can be distributed
 - Perform one specific task (e.g., filter, map)

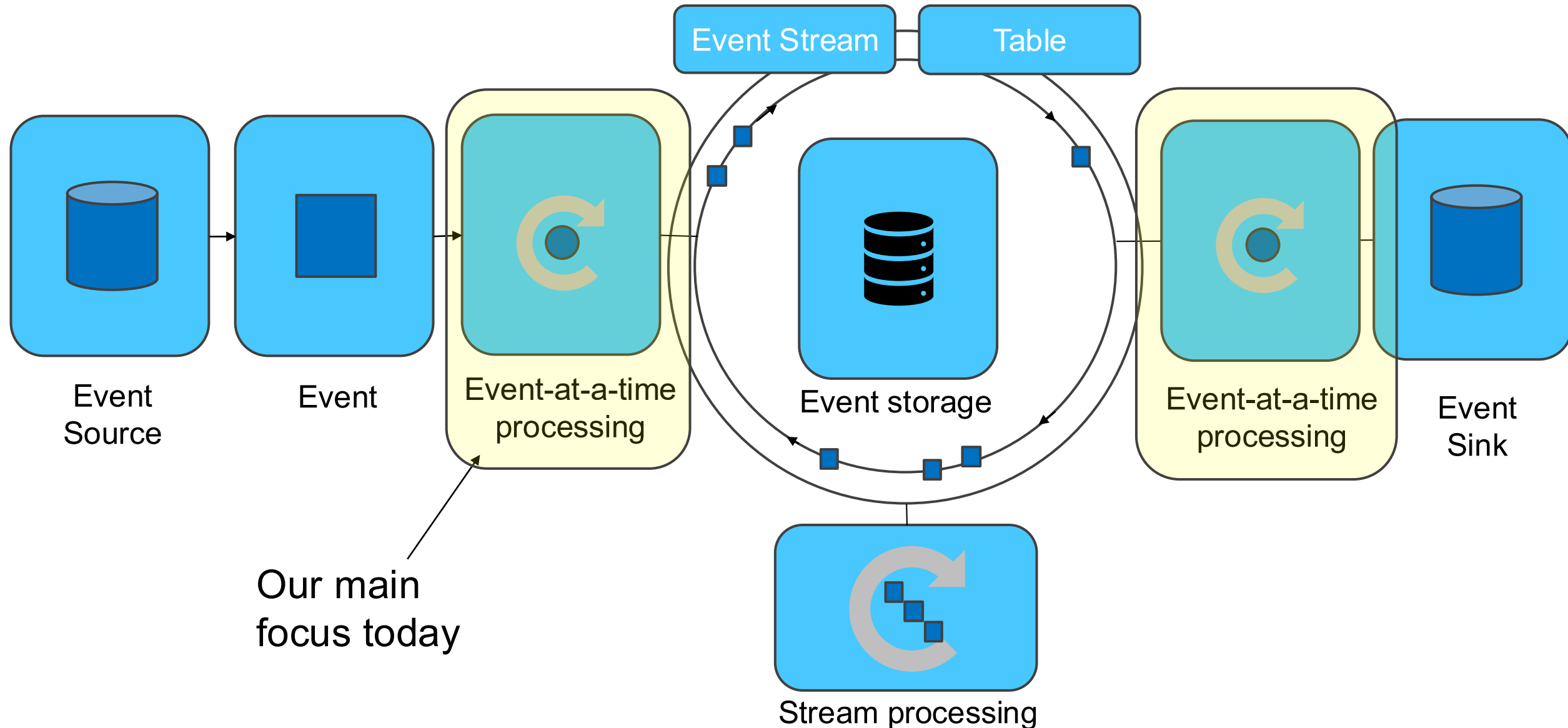


Event Processing Application

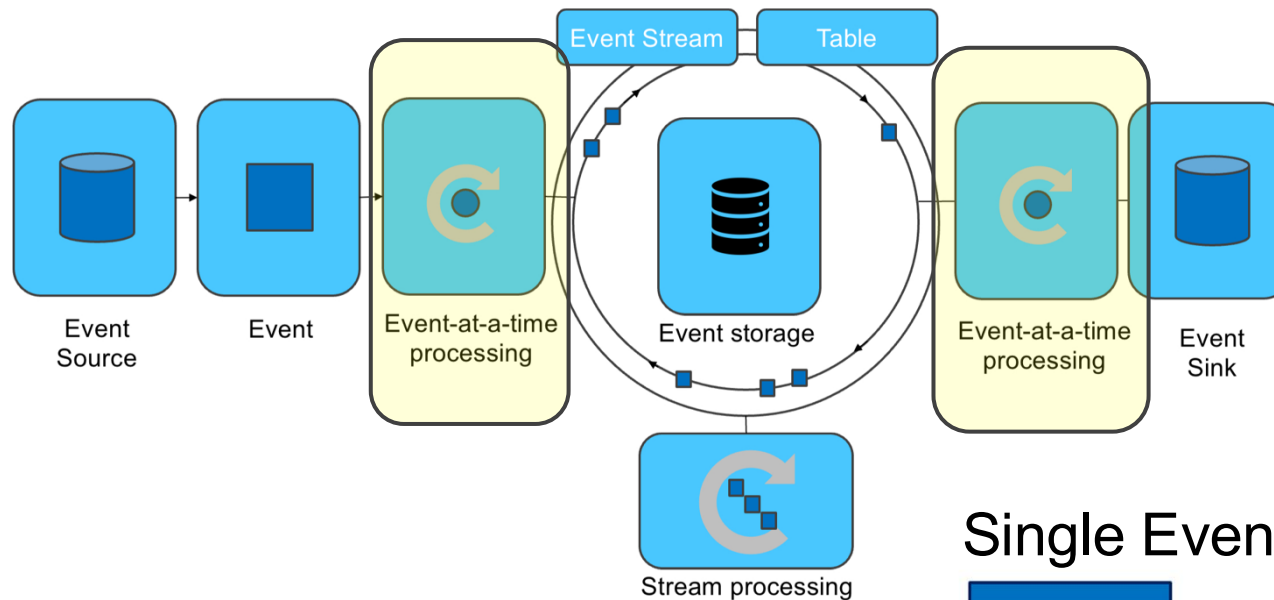
To build a fully-fledged application for data in motion we build an Event Processing Application by composing one or more Event Processors into an interconnected processing topology.



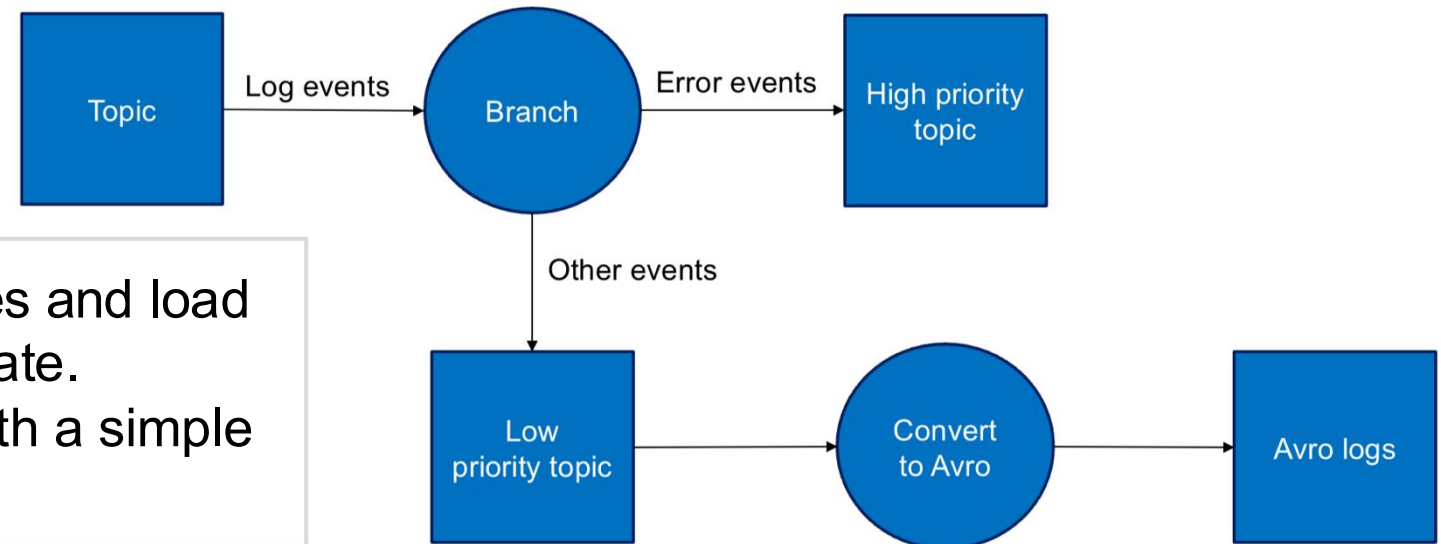
Event Streaming Platform



Single-Event Processing

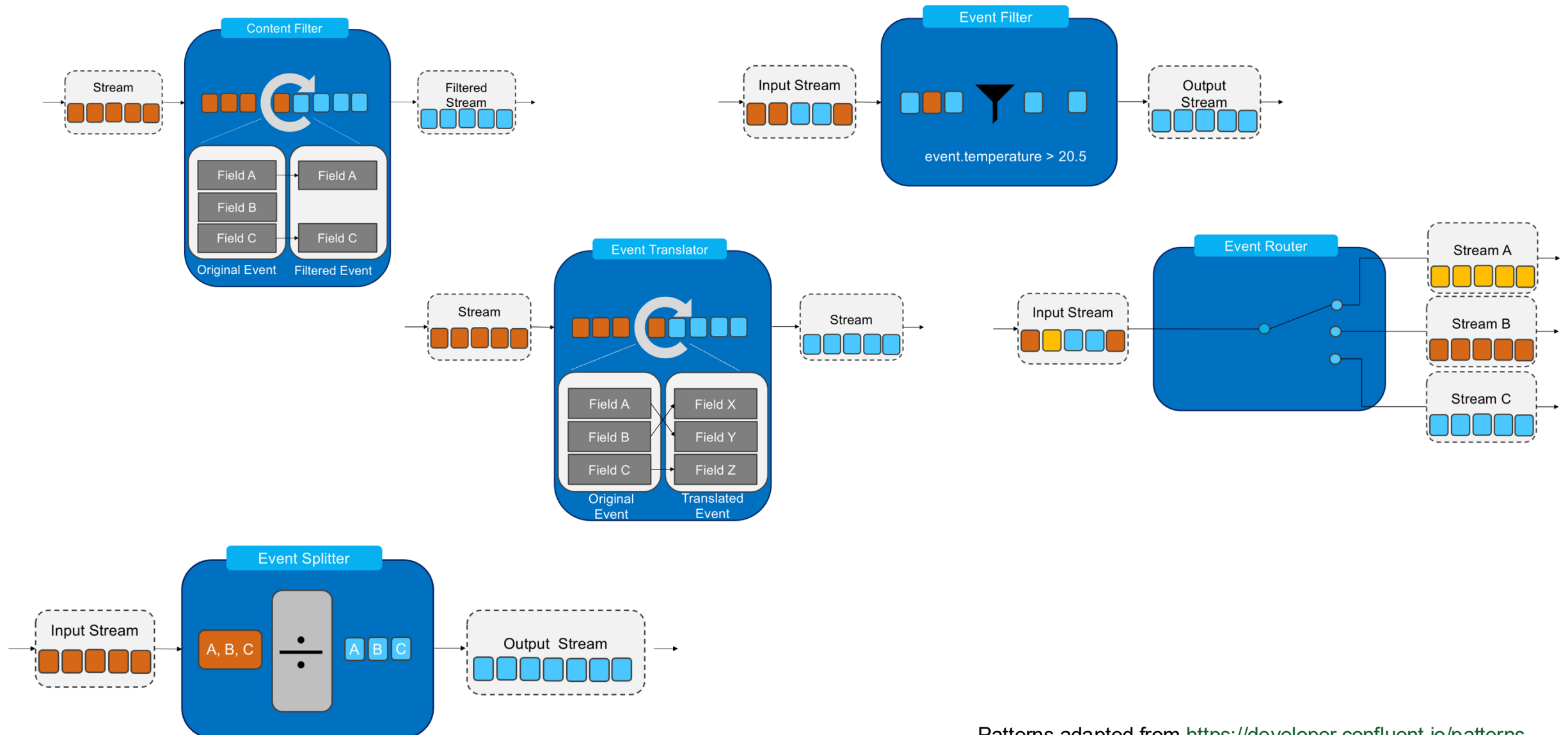


Single Event Processing Design Pattern



Easy recovery from application failures and load balancing due to the lack of state.
This pattern can be easily handled with a simple producer and consumer.

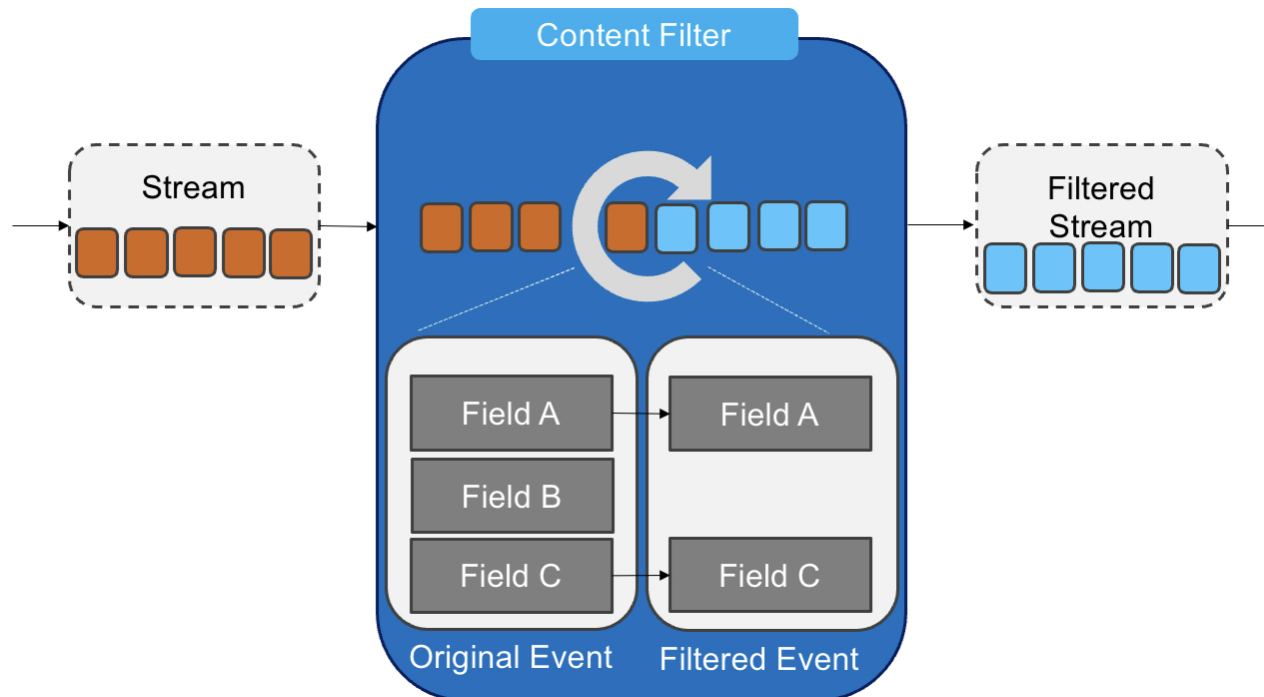
Types of Single Event Processors



Patterns adapted from <https://developer.confluent.io/patterns>

Content Filter.

- One input and one output stream
- Keeps all events
- Reduces each event to a subset of attributes
- Attribute values unchanged



Pattern related to [Content Filter](#) in *Enterprise Integration Patterns*, by Gregor Hohpe and Bobby Woolf.

```
builder
    .stream("temperature", Consumed.with(Serdes.Long(), readingSerde))
    .mapValues (
        (reading) -> {
            DeviceTemperature devTemp = new DeviceTemperature();

            devTemp.setDeviceID(reading.getDeviceID());
            devTemp.setTemperature(reading.getTemperature());

            return devTemp;
        })
    .to("deviceTemp", Consumed.with(Serdes.Long(), deviceTempSerde));
```

Note: In KStreams the implementation of both content filter as well as event translation both use `map` / `mapValues`.

Alternative (SQL-style Stream Processing) Content Filter with ksqlDB

- Using a `SELECT` statement in ksqlDB a rich event can be transformed into a simpler event with only a subset of the attributes
- By default the output stream will be partitioned like the input stream. Using `PARTITION BY` the data can be repartitioned.

Syntax:

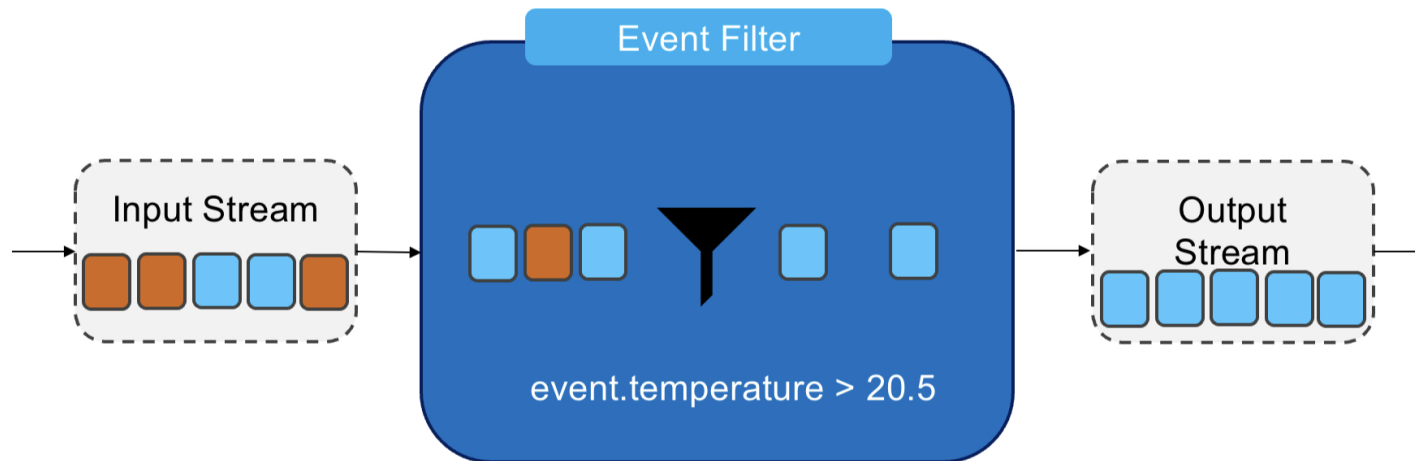
```
CREATE STREAM name of output stream [WITH (property_name = expression)] AS  
SELECT list of attributes  
FROM name of input stream
```

```
CREATE STREAM temperature (deviceId BIGINT KEY, roomNo INT, temperature DOUBLE)  
  WITH (kafka_topic = 'temperature', partitions = 1, value_format = 'json');
```

```
CREATE STREAM deviceTemp AS  
  SELECT deviceId, temperature  
  FROM temperature;
```


Event Filter. Only performs filter operations

- Has one input and one output stream
- Events fulfilling the filtering expression will be forwarded to the output stream (or dropped)
- Has no transformation capabilities
- Does not change or duplicate events



Pattern related to [Message Filter](#) in *Enterprise Integration Patterns*, by Gregor Hohpe and Bobby Woolf.

- Filtering selects a subset of events, while ignoring the rest.
- In Kstreams:
 - `filter(predicate)` forwards events where predicate = true
 - `filterNot(predicate)` drops events where predicate = true

```
builder
    .stream("temperature")
    .filter((key, reading) -> reading.temperature >= 20.5)
    .to("high_temperature");
```

Alternative (SQL-style Stream Processing) Event Filter with ksqlDB

Filtering involves selecting a subset of events to be processed, while ignoring the rest

Syntax:

```
CREATE STREAM name of output stream [WITH (property_name = expression)] AS
  SELECT column_name(s)
  FROM from_stream | from_table
  WHERE column_name operator value
```

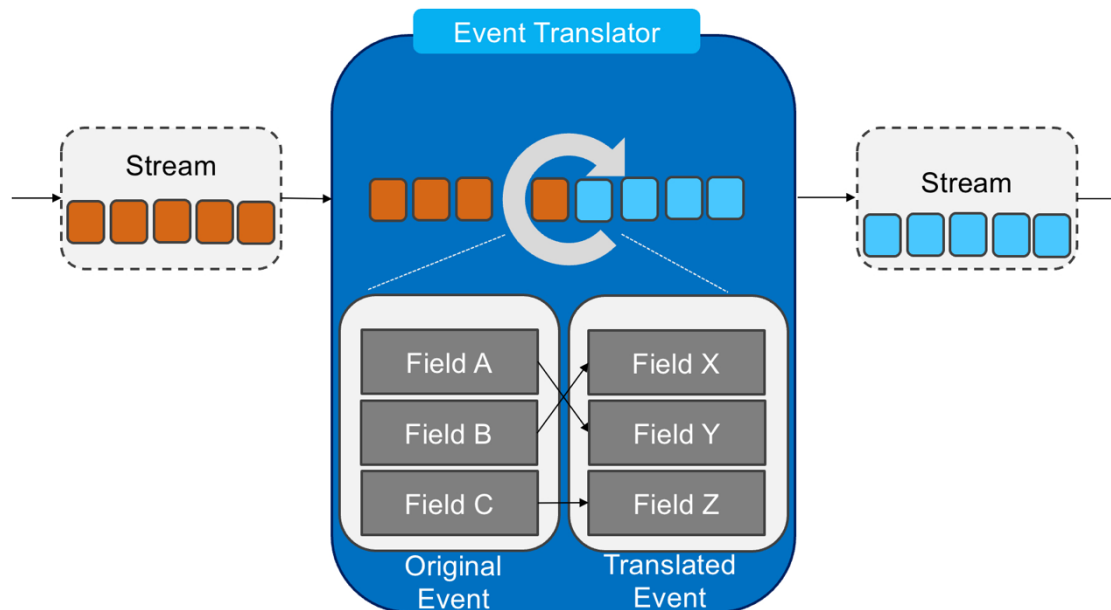
```
CREATE STREAM temperature (deviceId BIGINT KEY, roomNo INT, temperature DOUBLE)
  WITH (kafka_topic = 'temperature', partitions = 1, value_format = 'json');
```

```
CREATE STREAM high_temp WITH (kafka_topic = 'high_temperature') AS
  SELECT *
  FROM temperature
  WHERE temperature >= 20.5;
```

Tutorial: <https://developer.confluent.io/tutorials/filter-a-stream-of-events/ksql.html>

Event Translation

- One input, one output stream
- Modifies attribute values
- May change event structure
- No events added or removed



Pattern related to [Message Translation](#) in *Enterprise Integration Patterns*, by Gregor Hohpe and Bobby Woolf.

To set and modify attribute values and their types the operator `mapValues` can be used.

```
builder
    .stream("input-topic")
    .mapValues ( (v) -> v.toUpperCase() )
    .to("output-Topic") ;
```

Note: In KStreams the implementation of both content filter as well as event translation both use `map` / `mapValues`.

Alternative (SQL-style Stream Processing) Event Translation in ksqlDB

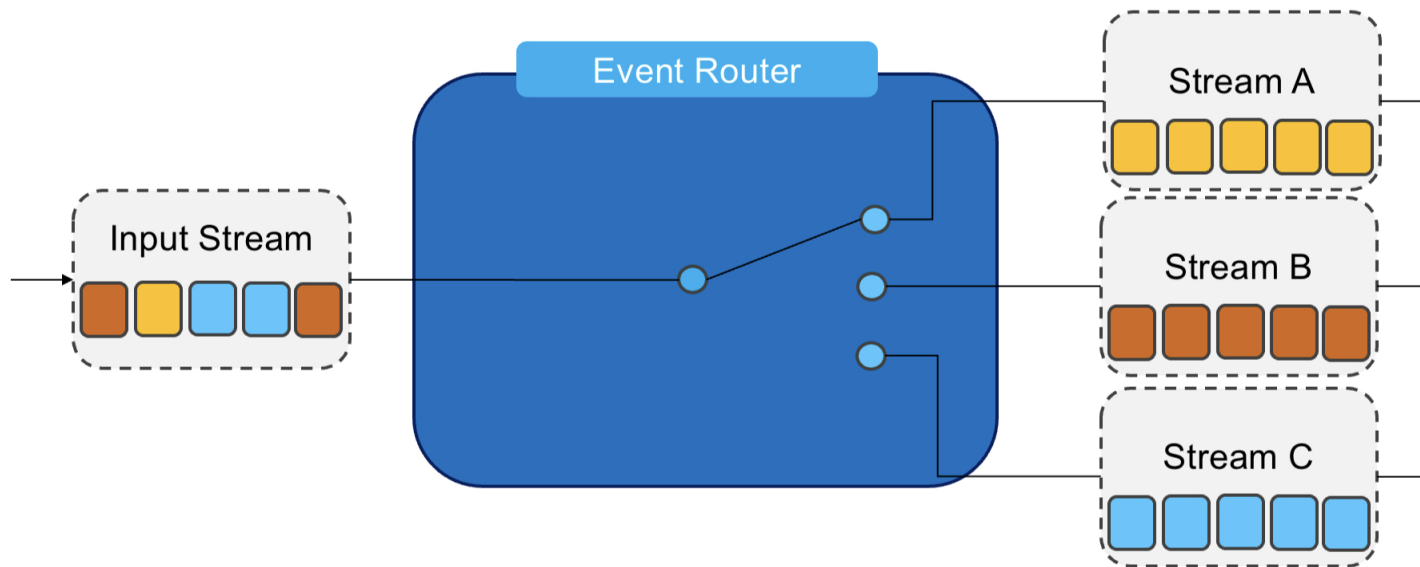
To set and modify attribute values ksqlDB supports the usage of different mathematical and logical operators as well as functions.

```
CREATE STREAM temperature (deviceId BIGINT KEY, roomNo INT, temperature DOUBLE)  
  WITH (kafka_topic = 'temperature', partitions = 1, value_format = 'json');  
  
CREATE STREAM outputStream AS  
  SELECT deviceId, roomNo, temperature * 9/5 + 32 as temp  
  FROM temperature;
```

The temperature value is
translated into Fahrenheit

Event Router. Splits the input stream into several output streams

- One input, multiple output streams
- Routes events based on conditions
- Does not modify events
- Does not duplicate events



Pattern related to [Message Router](#) in *Enterprise Integration Patterns*, by Gregor Hohpe and Bobby Woolf.

One possibility to implement an event router is to use the `branch` operator. Branching decisions are based on supplied `Predicates`, which need to be fixed in number. The output topics need to be created ahead of running the application.

```
StreamsBuilder builder = new StreamsBuilder();
KStream<Long, Reading> readings = builder.stream("temperature");

final KStream<Long, Reading>[] branches = readings.branch(
    (id, reading) -> reading.getTemperature() >= 20.5,
    (id, reading) -> reading.getTemperature() < 19);
branch[0].to("high_temperature");
branch[1].to("low_temperature");
```

Note: `branch` is deprecated and replaced by `split`, see [here](#).

Lab12P1 uses `split`, Lab12P2 uses `branch`.

Blog Entry: <https://www.confluent.io/blog/putting-events-in-their-place-with-dynamic-routing/>

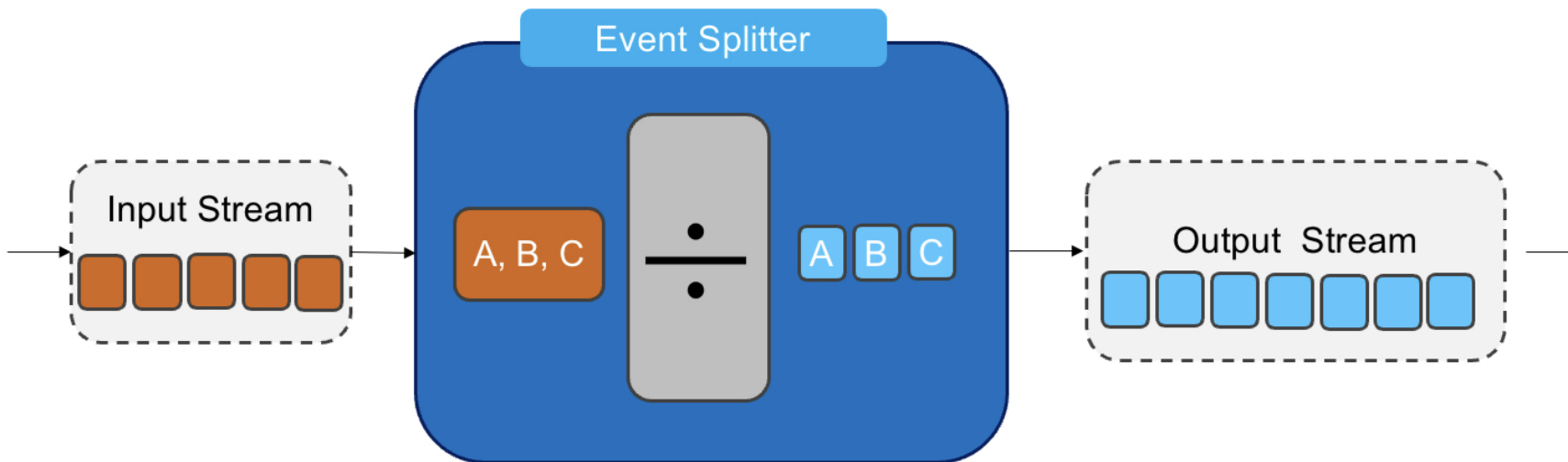
Alternative (SQL-style Stream Processing) Event Router with ksqlDB

Incoming events are passed to several output streams using the `CREATE STREAM` syntax with proper `WHERE` statements.

```
CREATE STREAM temperature (deviceId BIGINT KEY, roomNo INT, temperature DOUBLE)  
  WITH (kafka_topic = 'temperature', partitions = 1, value_format = 'json');  
  
CREATE STREAM high_temp WITH (kafka_topic = 'high_temperature') AS  
  SELECT *  
    FROM temperature  
    WHERE temperature >= 20.5;  
  
CREATE STREAM low_temp WITH (kafka_topic = 'low_temperature') AS  
  SELECT *  
    FROM temperature  
    WHERE temperature < 19;
```

Event Splitter. Splits the original events into child events and publishes one event per child.

- Has one input and one output stream
- Can split events into zero or more child events which can then be processed separately
- This can result into changes of the key



Pattern related to [Event Splitter](#) in *Enterprise Integration Patterns*, by Gregor Hohpe and Bobby Woolf.

Event Splitter in KStreams

To split an event into several child events without changing the key `flatMapValues` can be used. Values and value types can be modified. The advantage over `flatMapValues` over `flatMap` is that it does not cause data re-partitioning.

```
builder
    .stream("sentences")
    .flatMapValues((k, v) ->
        Array.asList(v.toLowerCase().split(" ")))
    .to("words");
```

Alternative (SQL-style Stream Processing) Event Splitter in ksqlDB

To split an event into several child events the built-in table function `EXPLODE` can be used.

```
{sensor_id:12345 readings: [23, 56, 3, 76, 75]}  
{sensor_id:54321 readings: [12, 65, 38]}
```

```
CREATE STREAM exploded_stream AS  
  SELECT sensor_id, EXPLODE(readings) as reading  
  FROM batched_readings;
```

```
{sensor_id:12345 reading: 23} {sensor_id:12345 reading: 56} {sensor_id:12345 reading: 3}  
{sensor_id:12345 reading: 76} {sensor_id:12345 reading: 75} {sensor_id:54321 reading: 12}  
{sensor_id:54321 reading: 65} {sensor_id:54321 reading: 38}
```

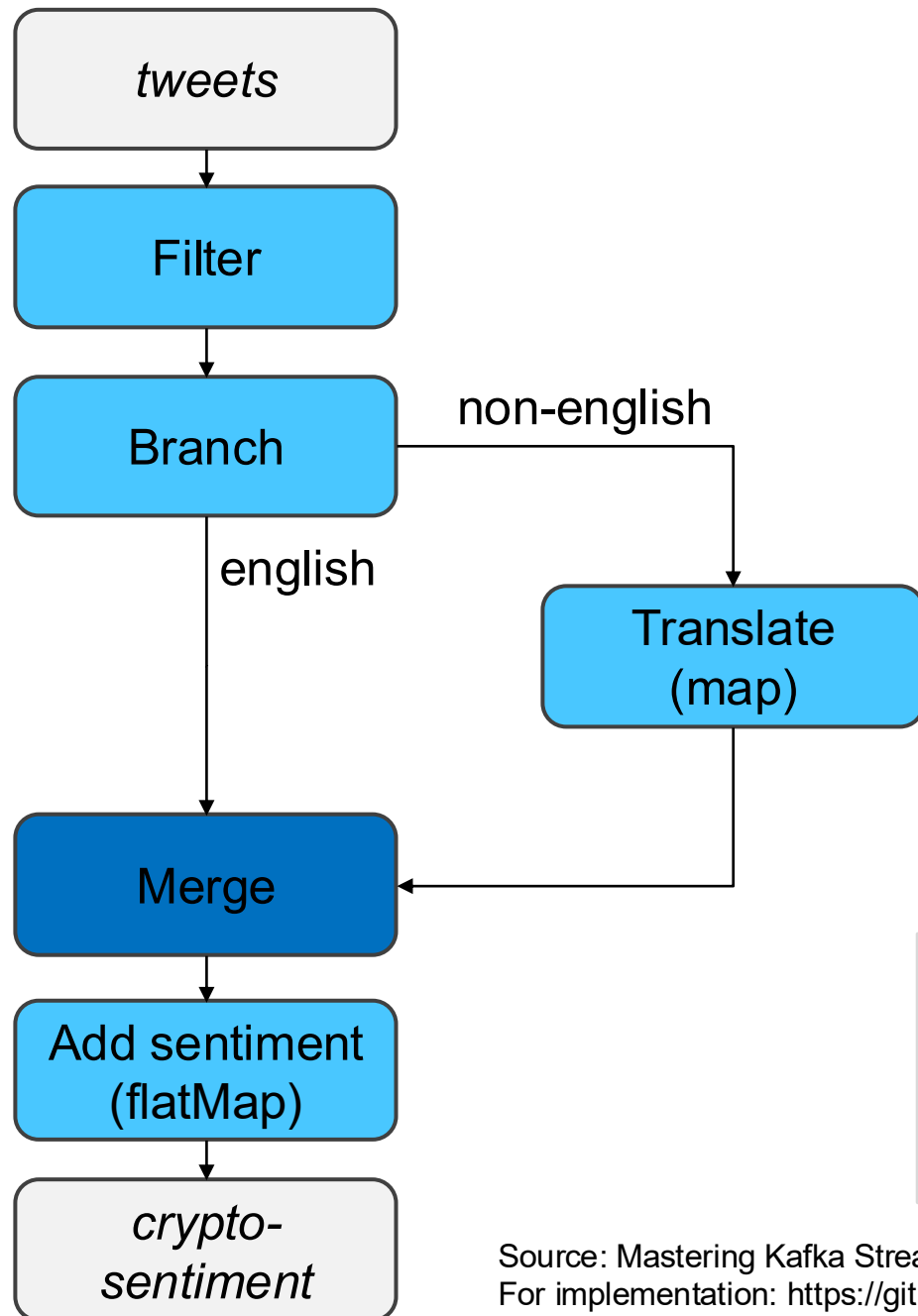
Example Topology Stateless Processing

Event Filter

Event Router

Stream Merger

Event Splitter



Event Translator

 Single Event Processor
 Event Stream Processor

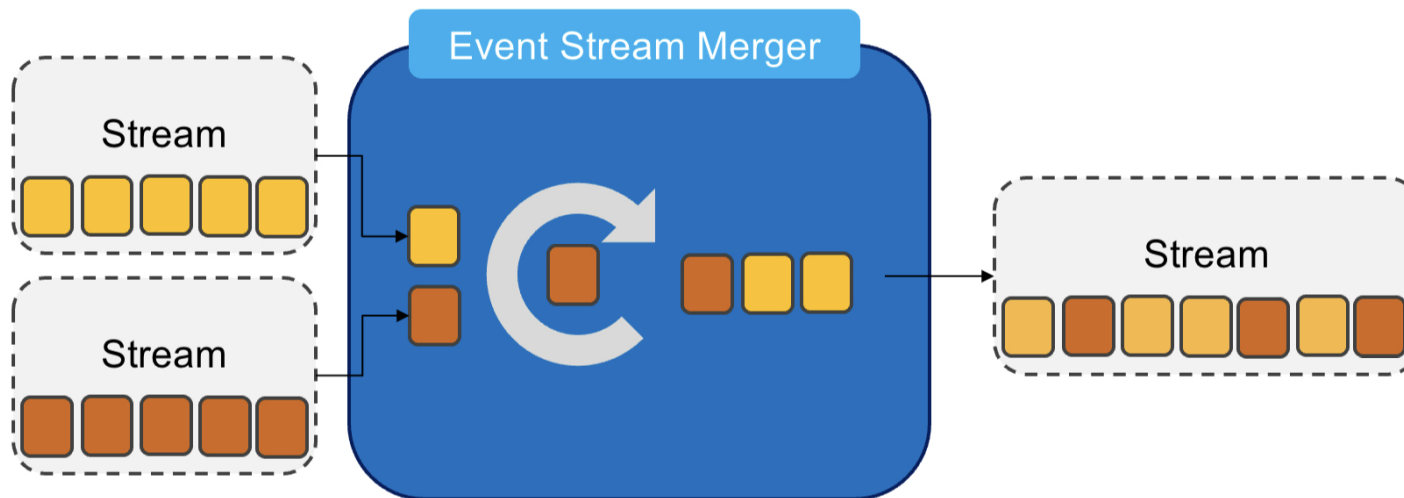
Source: Mastering Kafka Streams and ksqlDB
For implementation: <https://github.com/mitch-seymour/mastering-kafka-streams-and-ksqldb>

Event Stream Merger

Example of a Stateless Stream Processor

Event Stream Merger. Merges several event streams into a single stream

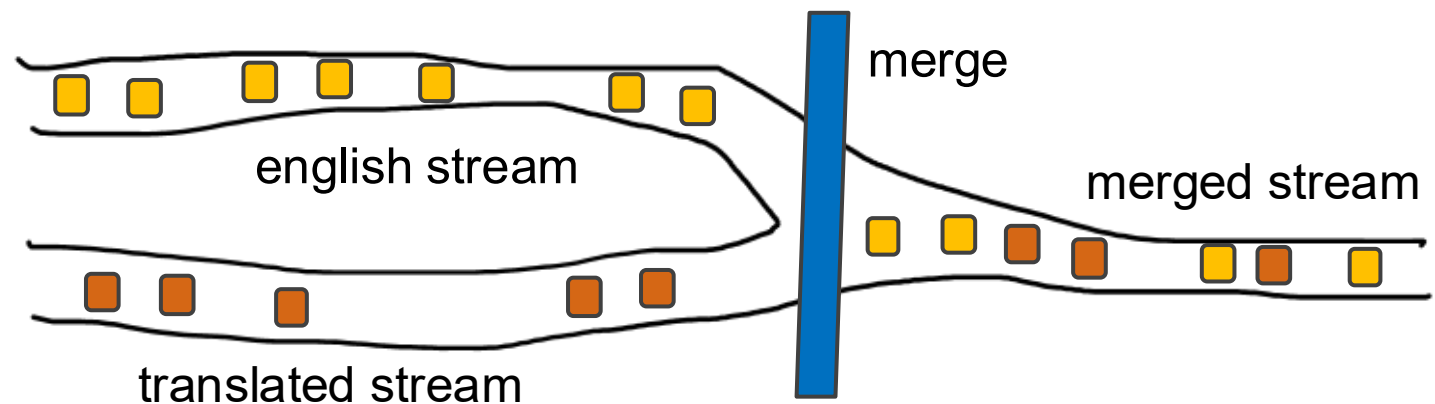
- Multiple input streams, one output stream
- Events are forwarded unchanged
- No ordering guarantees across streams



Event Stream Merger in KStreams

- To merge multiple streams into one the `merge` operator can be used.
- Kafka does not provide any guarantees on the processing orders of the underlying streams
- Streams can only be merged if their events have the same key and value types

```
KStream<byte[], Tweet> mergedStream = englishStream.merge(translatedStream);
```



[Tutorial: How to merge streams with KStreams](#)

Alternative (SQL-style Stream Processing)

Event Stream Merger in ksqlDB

- To merge multiple streams into one `insert into` can be used.
- Kafka does not provide any guarantees on the processing orders of the underlying streams
- Streams can only be merged if their events have the same key and value types

```
CREATE STREAM rock_songs (artist VARCHAR, title VARCHAR)
  WITH (kafka_topic='rock_songs', partitions=1, value_format='avro');

CREATE STREAM classical_songs (artist VARCHAR, title VARCHAR)
  WITH (kafka_topic='classical_songs', partitions=1, value_format='avro');

CREATE STREAM all_songs (artist VARCHAR, title VARCHAR, genre VARCHAR)
  WITH (kafka_topic='all_songs', partitions=1, value_format='avro');

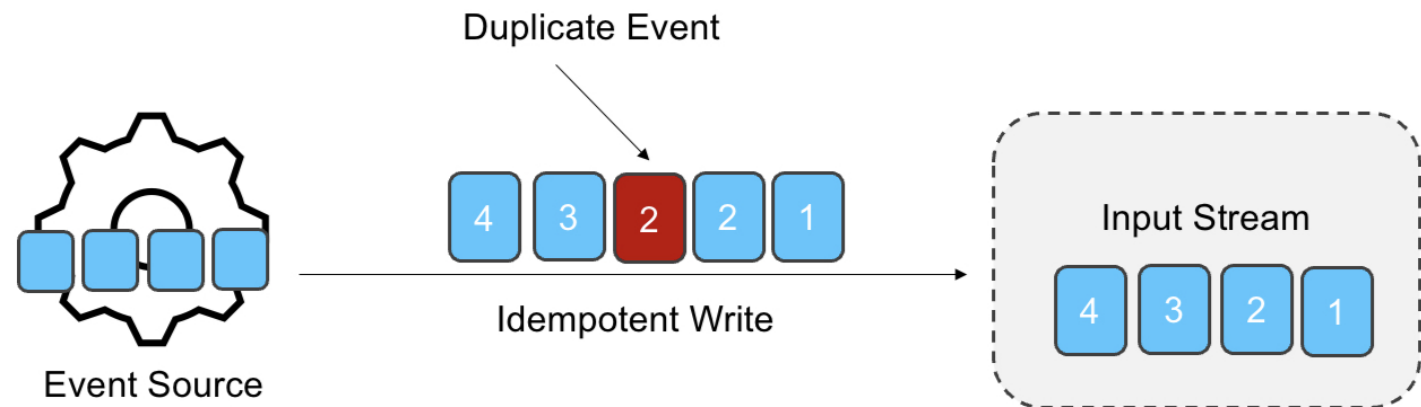
INSERT INTO all_songs SELECT artist, title, 'rock' AS genre FROM rock_songs;

INSERT INTO all_songs SELECT artist, title, 'classical' AS genre FROM classical_songs;
```

[Tutorial: How to merge streams with ksqlDB](#)

Idempotent Writer

- Retries may produce duplicate events
- Idempotent writer ensures duplicates are discarded
- Guarantees that each event is written once



Kafka configuration:

```
enable.idempotence=true
```

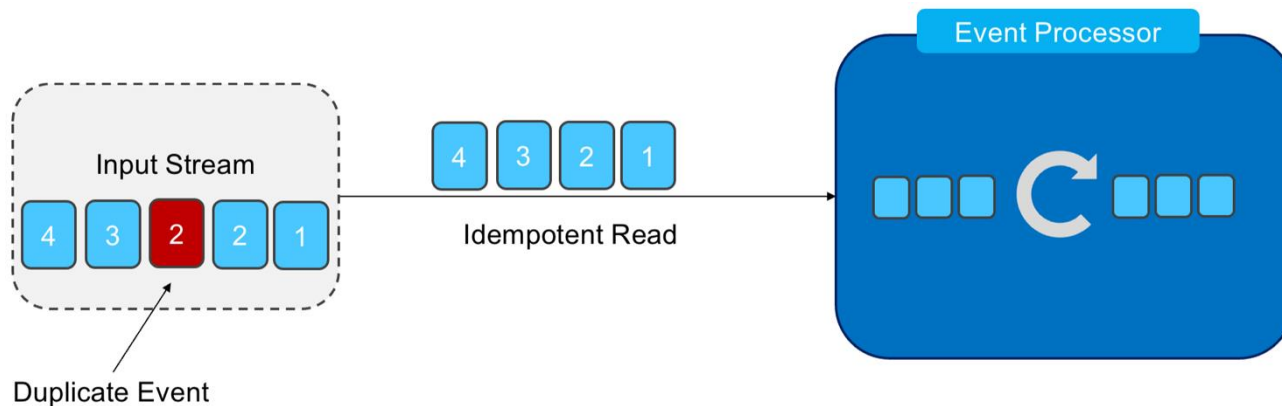
Exactly-once processing

```
processing.guarantee="exactly_once"
```

Source: https://www.confluent.io/blog/simplified-robust-exactly-one-semantics-in-kafka-2-5/?_ga=2.159185402.1110636579.1649877876-892569557.1649877876

Idempotent Reader

- Reader may see duplicate events
- Idempotent reader handles duplicates safely
- Ensures correct processing



Kafka configuration:

```
isolation.level="read_committed"
```

Exactly-once processing:

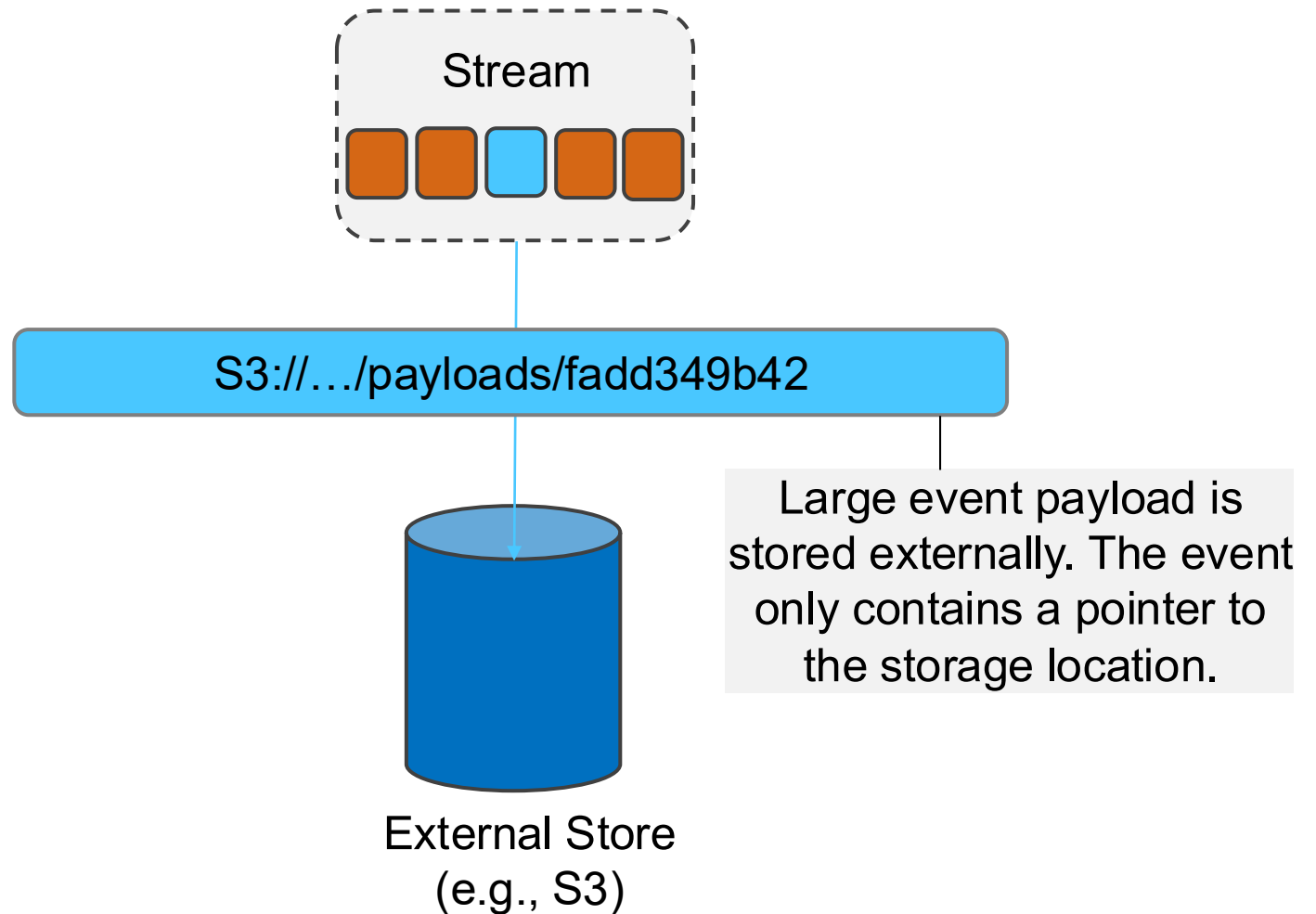
```
processing.guarantee="exactly_once"
```

Source: https://www.confluent.io/blog/simplified-robust-exactly-one-semantics-in-kafka-2-5/?_ga=2.159185402.1110636579.1649877876-892569557.1649877876

Two Patterns to Handle Large Events

Claim Check

- When events are too large to be moved through the event streaming platform, then the event payload can be stored externally in a datastore and only a pointer to the storage location (claim check) is sent.

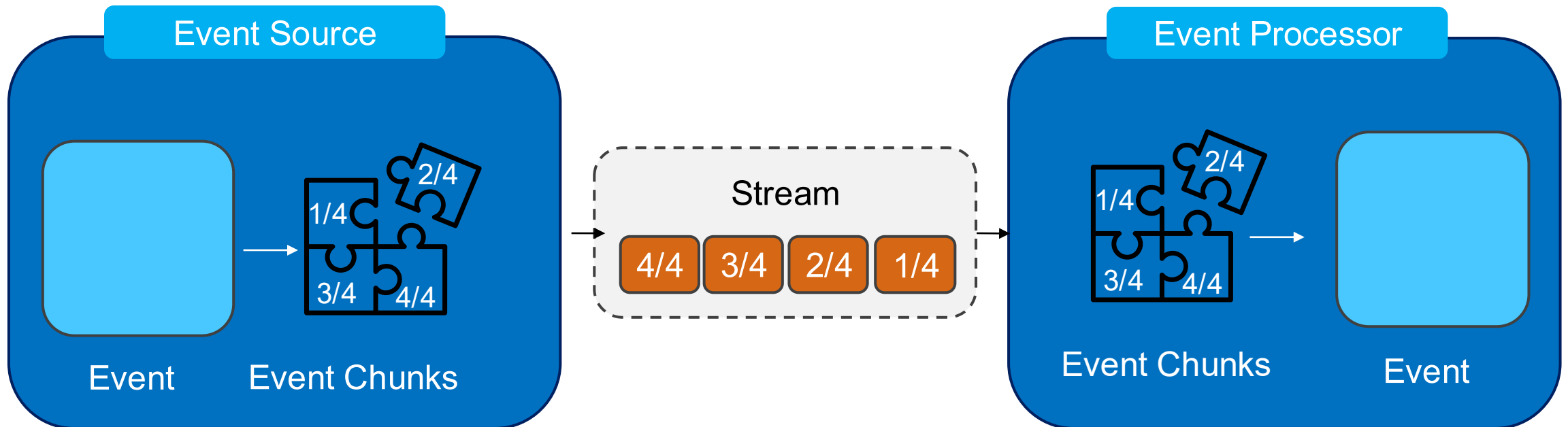


Pattern similar to [Claim Check](#) in *Enterprise Integration Patterns*, by Gregor Hohpe and Bobby Woolf.
Further resources: <https://dzone.com/articles/processing-large-messages-with-apache-kafka>

Two Patterns to Handle Large Events

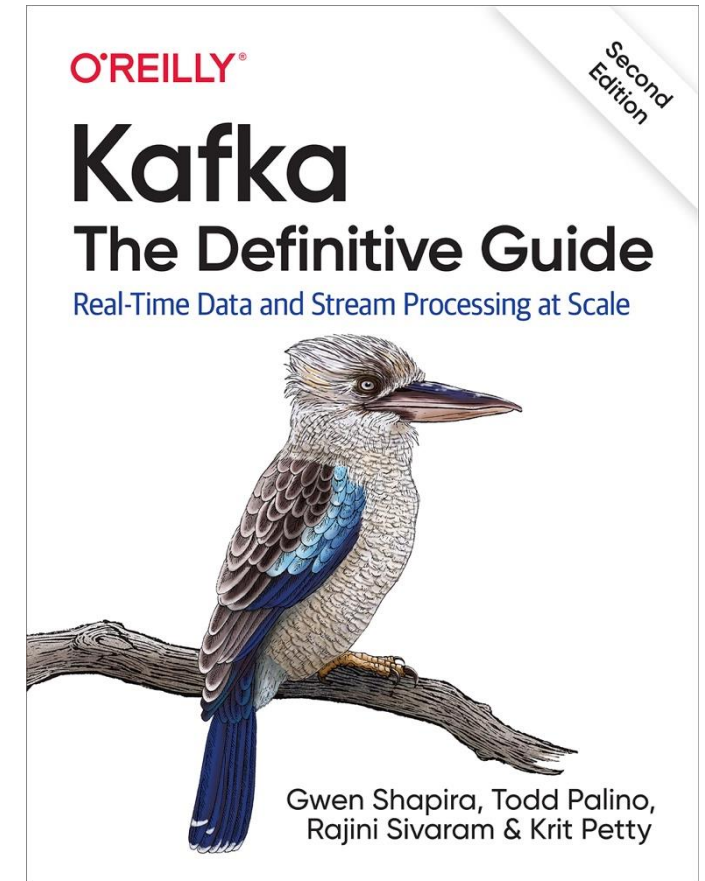
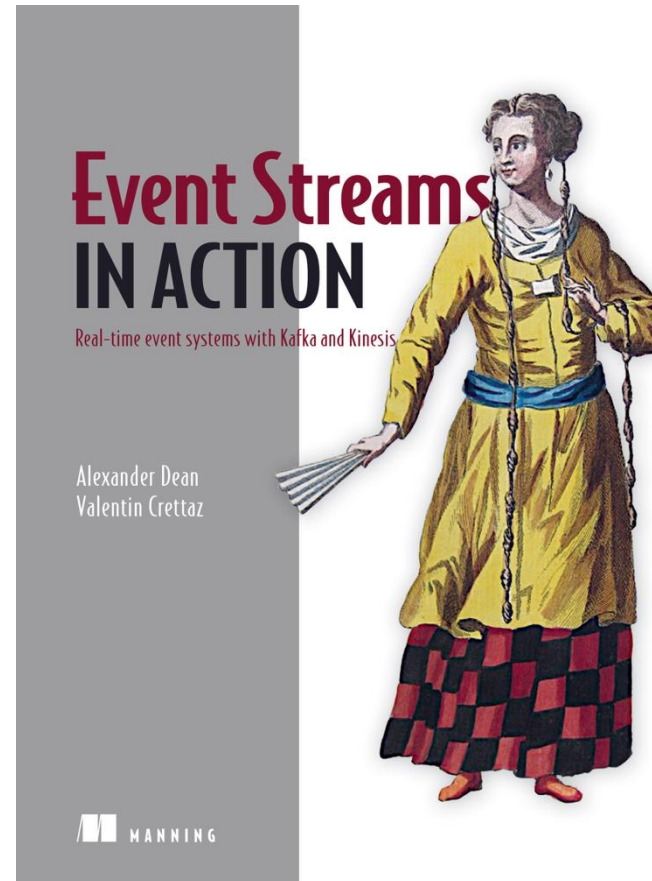
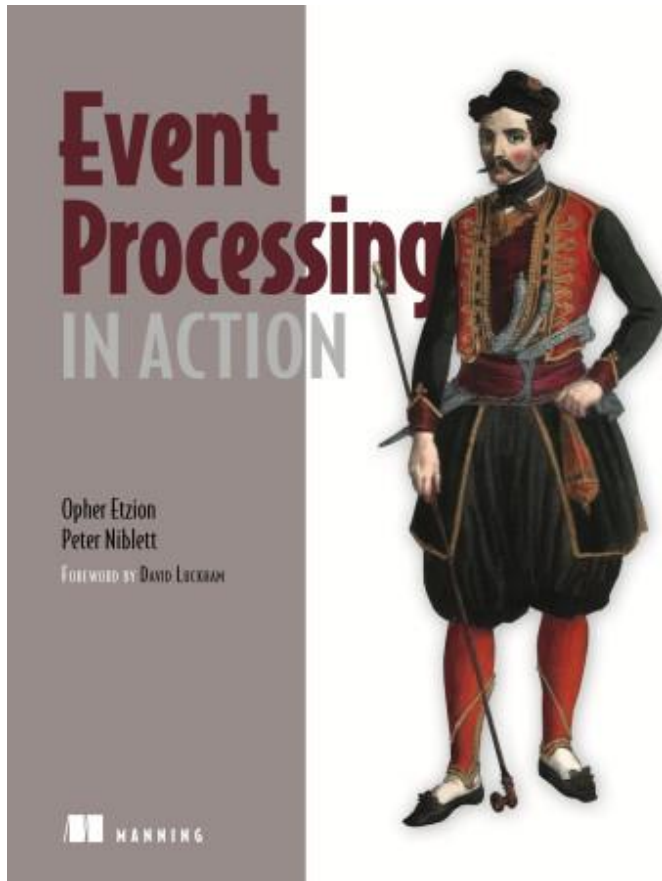
Event Chunking

- Instead of storing the event as a whole in the event processing platform, it can be split into several smaller events („chunking“), which are then sent across and re-assembled at the client side to restore the original larger event



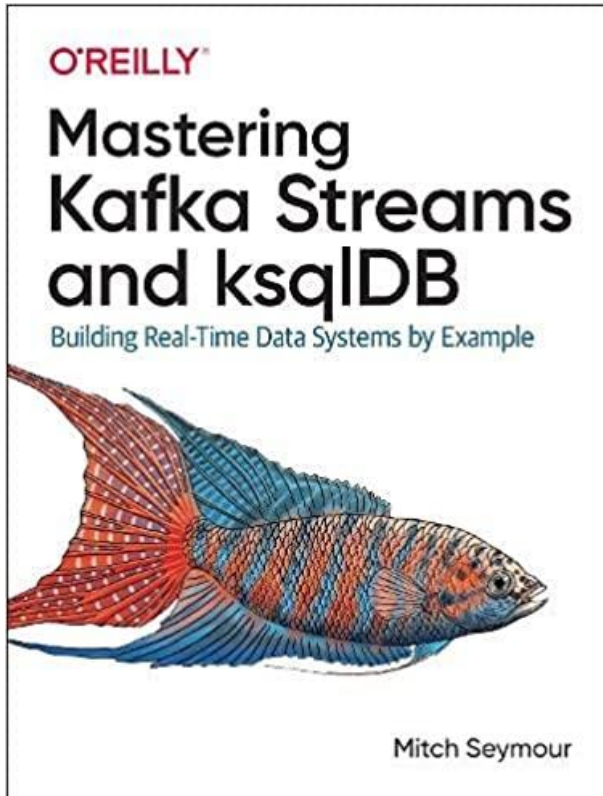
Further resources: <https://dzone.com/articles/processing-large-messages-with-apache-kafka>

References and Further Materials

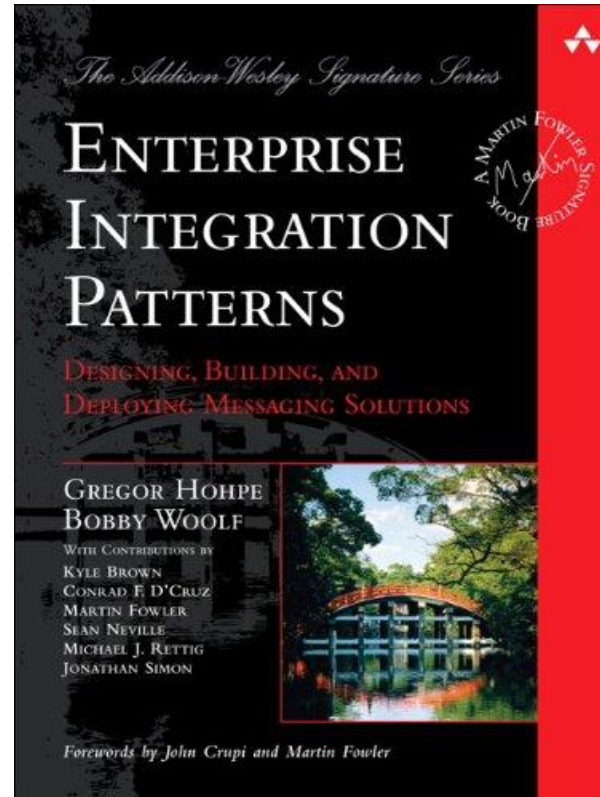


[Get free ebook](#)
via Confluent

Further Resources



[Get free ebook](#)
via Confluent

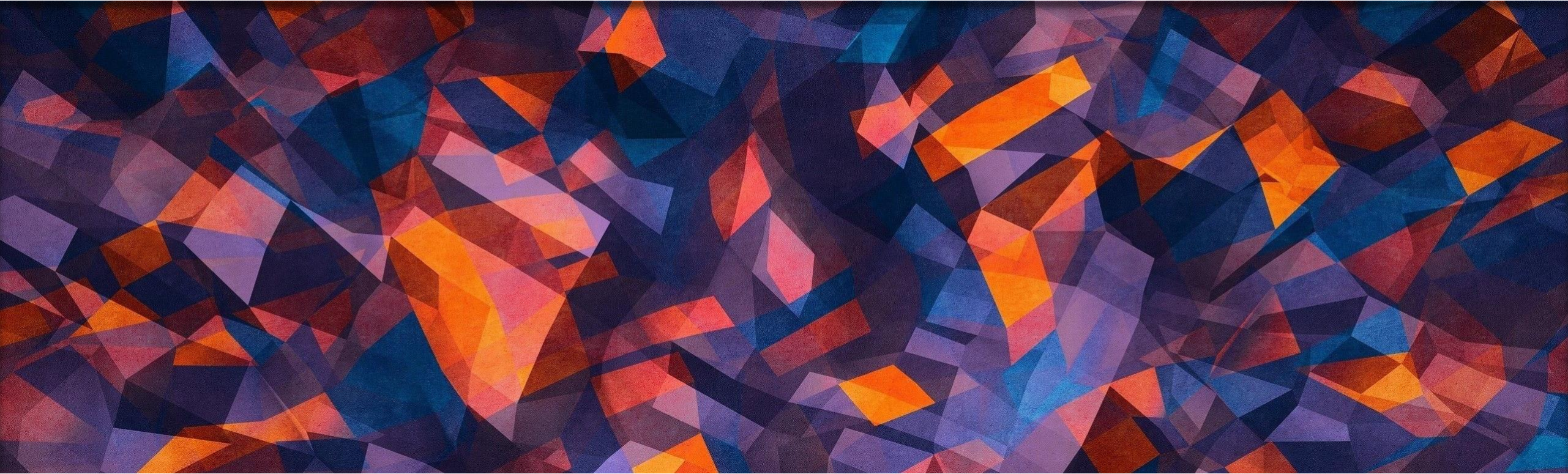


<https://www.enterpriseintegrationpatterns.com>

Event Streaming Patterns
<https://developer.confluent.io/patterns>



Universität St.Gallen



Event-driven and Process-oriented Architectures

Exercises

Lecture 8

- For Kafka Streams stateless:

- HELLO WORLD (aka YellingApp):

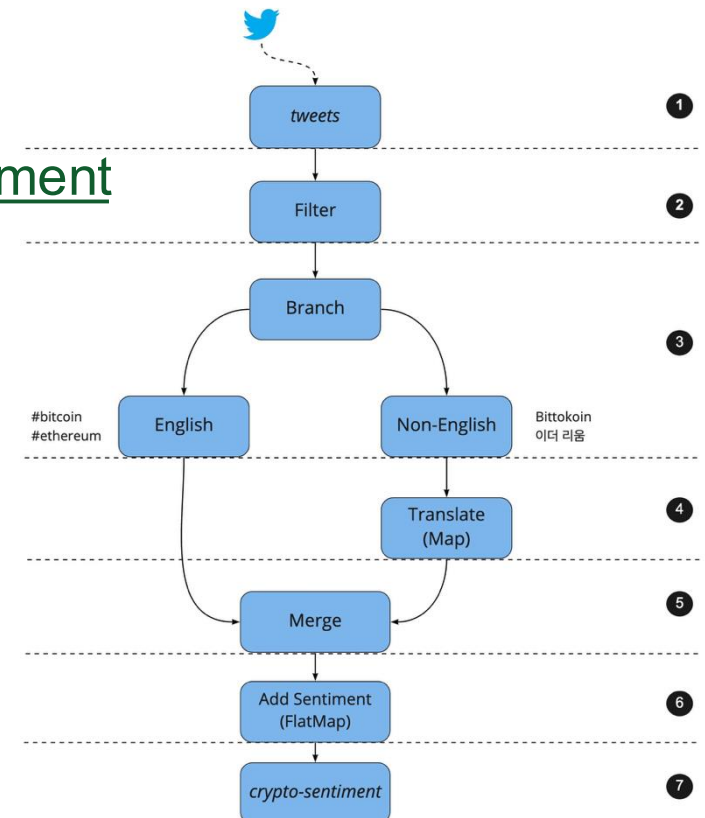
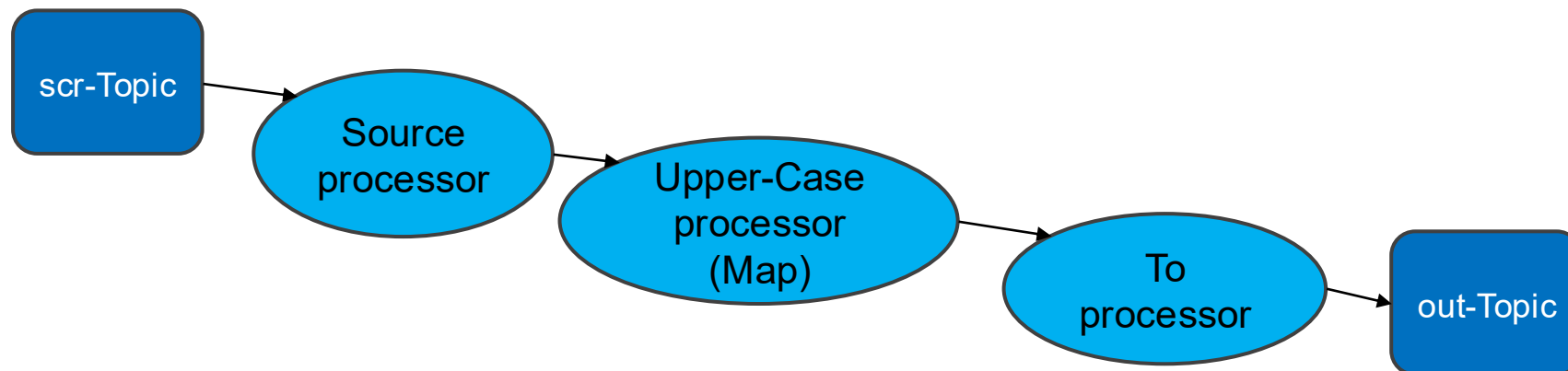
- <https://github.com/scs-edpo/lab11-kafka-streams-yellingapp>

- Chapter 2 in: Mastering Kafka Streams and ksqlDB and Chapter 3 in: Kafka Streams in Action

- CryptoSentiment:

- <https://github.com/scs-edpo/lab12Part1-kafka-streams-cryptosentiment>

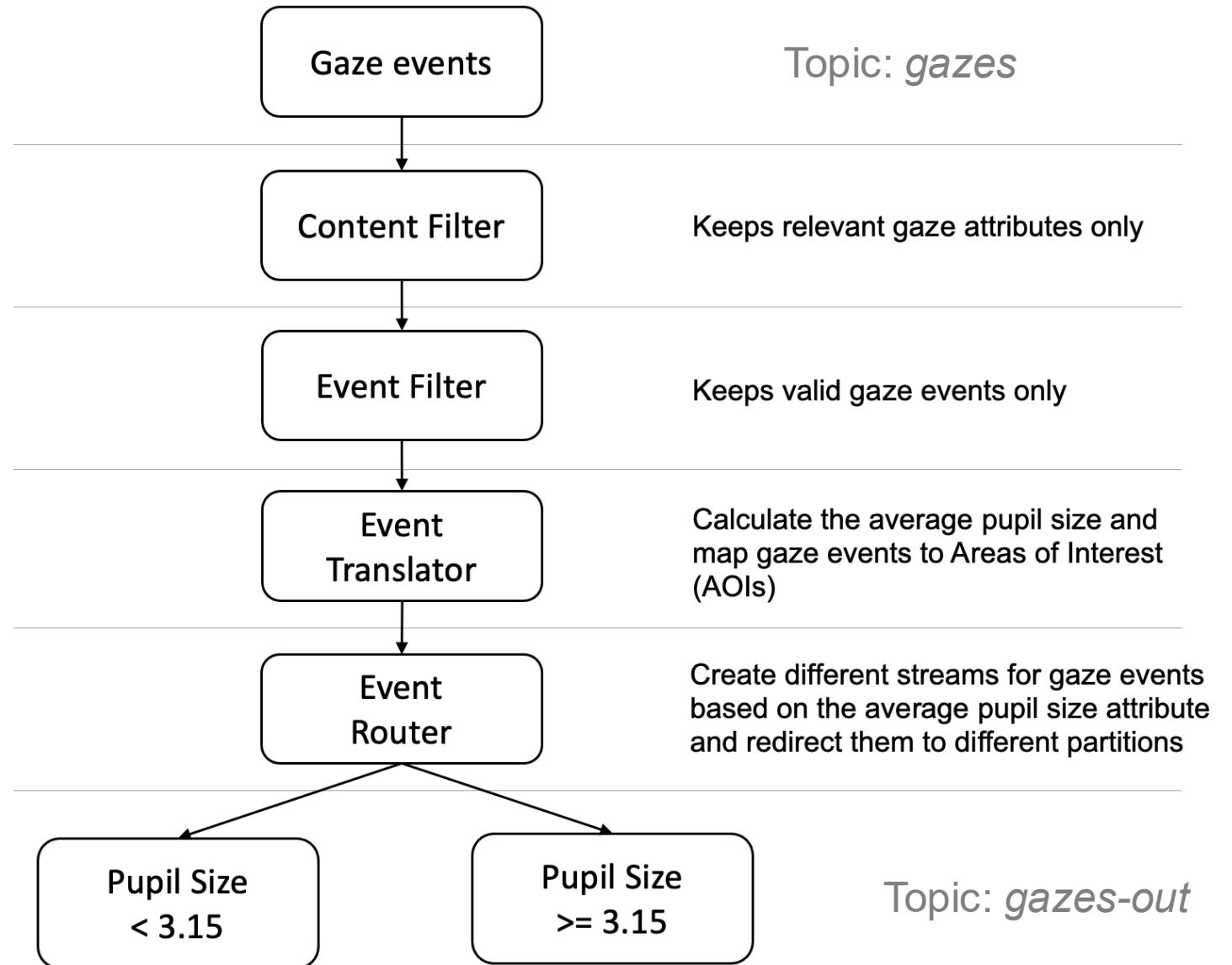
- Chapter 3 in: Mastering KStreams and ksqlDB



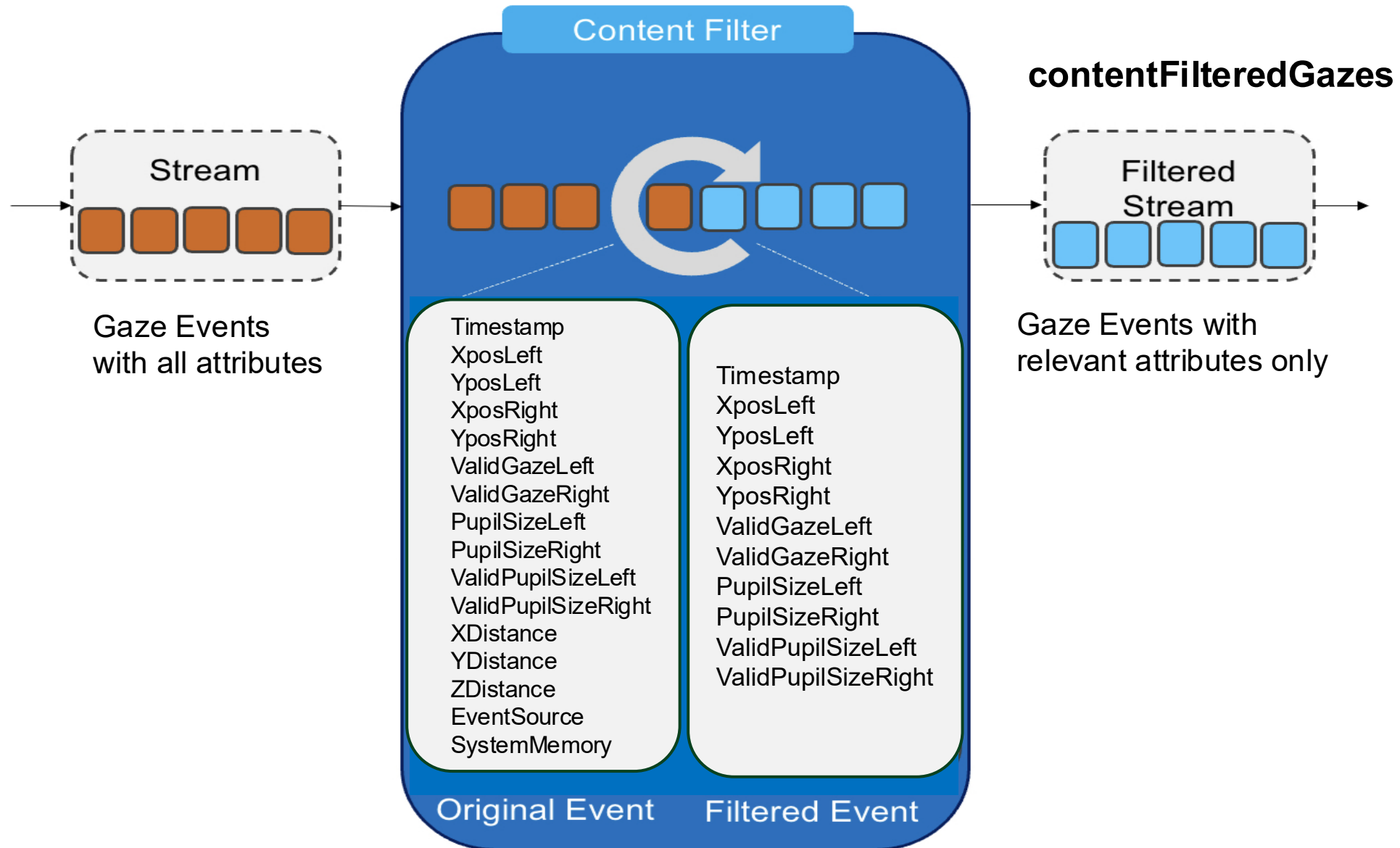
- Eye Tracking Events Processing (Part1)

<https://github.com/scs-edpo/lab12Part2-kafka-streams-eyeTracking1>

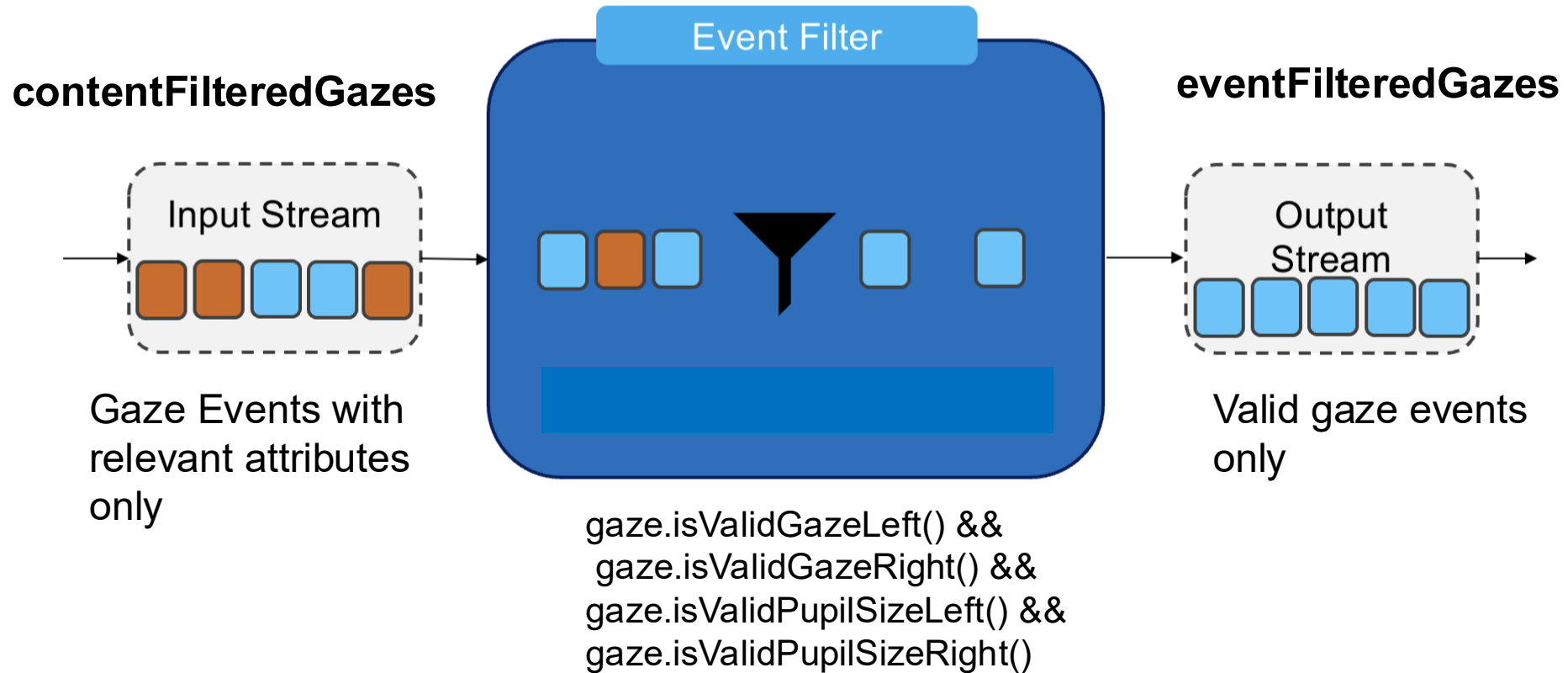
Overview of the topology

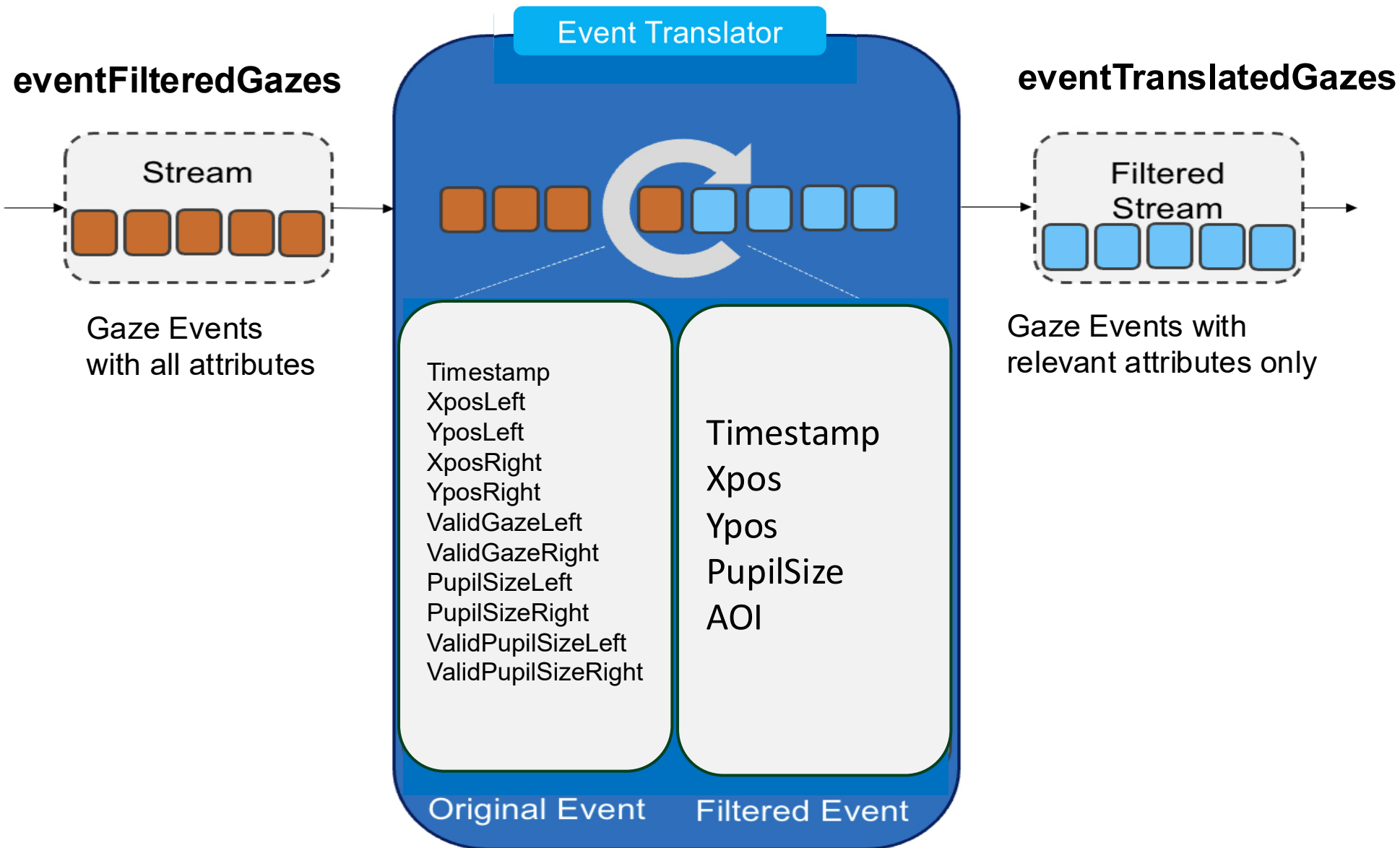


Eye Tracking Events Processing (Part1) - Topology



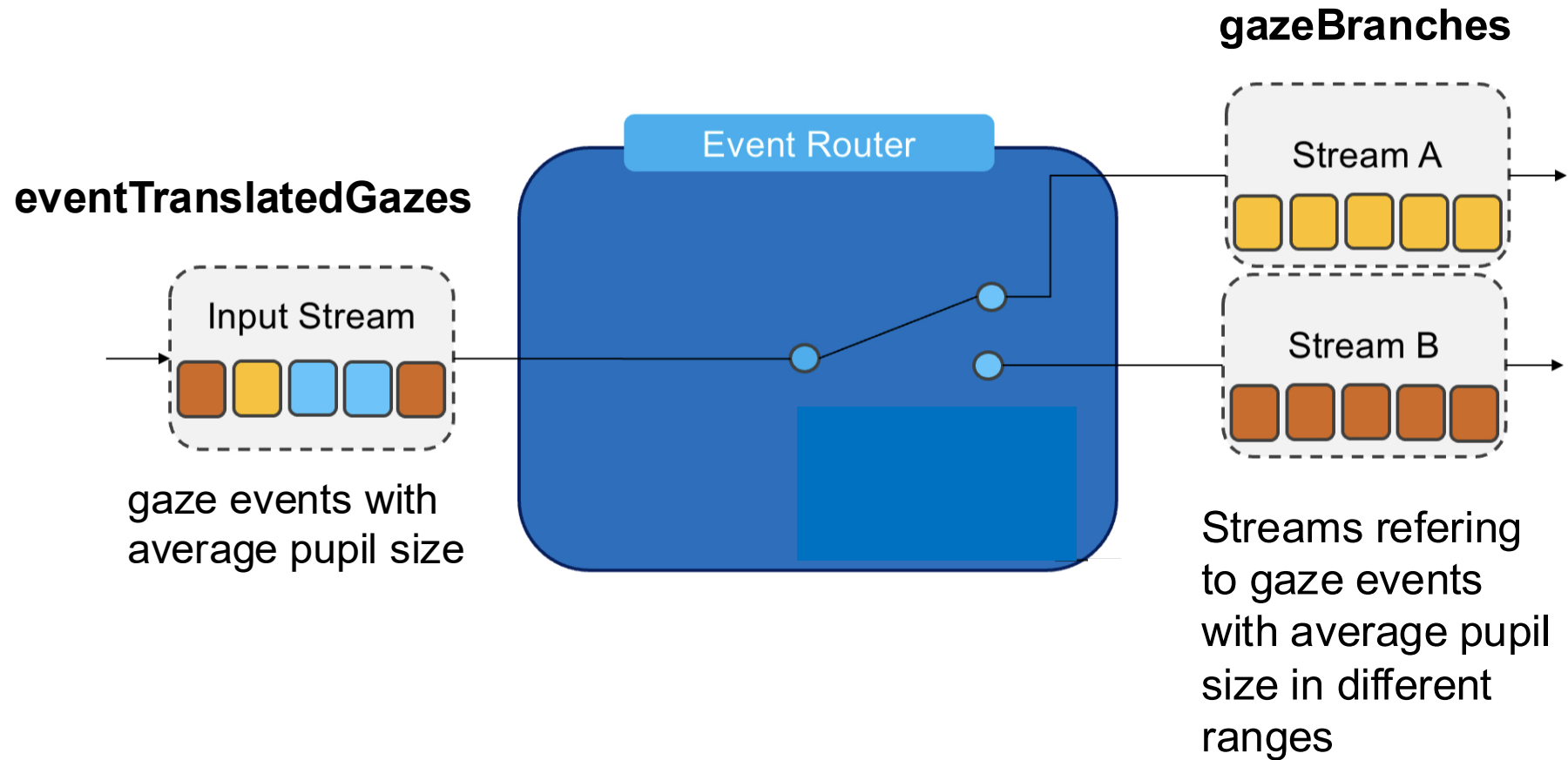
Eye Tracking Events Processing (Part1) - Topology





- Compute xpos and ypos from the left and right positions of gazes
- Compute average pupil size from the left and right pupil sizes
- Map the gaze x and y coordinates to the corresponding area of interest (AOI)

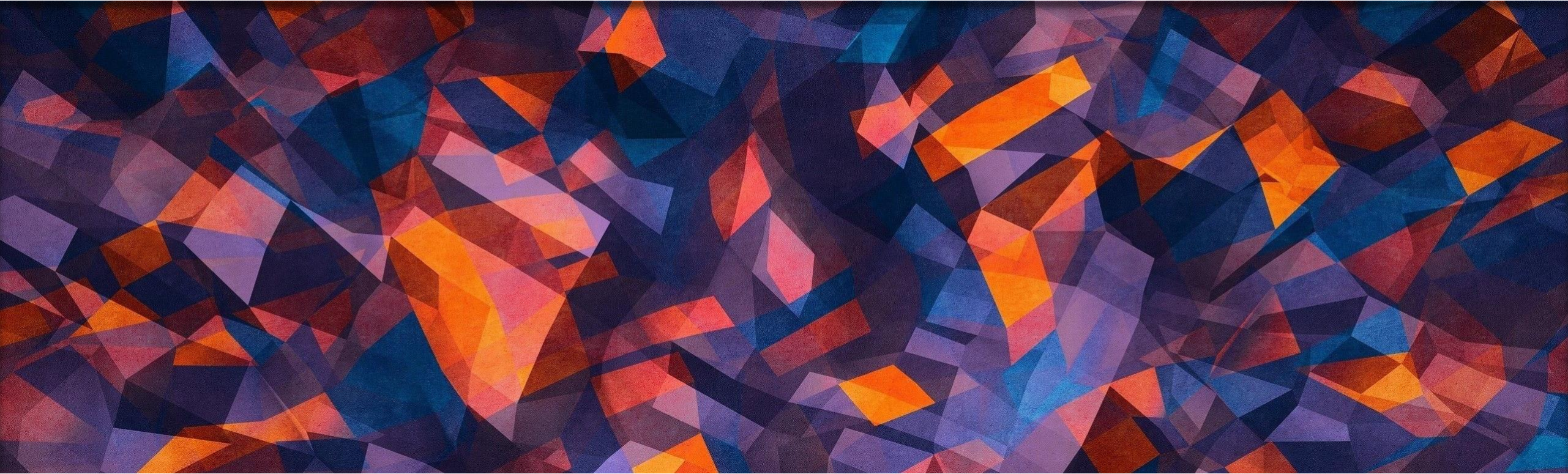
Eye Tracking Events Processing (Part1) - Topology



- Route the gaze events based on different pupil size ranges



Universität St.Gallen



Event-driven and Process-oriented Architectures

Project

Lecture 8

1. Develop one or several stream processing applications
2. The stream processing applications should include both stateless and stateful stream processing
 - At least several stateless operations (Week 8)
 - Usage of both streams and tables (Week 9)
 - Consider data from more than one stream (Week 9)
 - Interactive queries (Week 9)
 - Windowed operations (Week 10)
3. Describe for each developed stream processing application the topology in graphical form
4. Elaborate on the chosen patterns
5. Use Avro schema (registryless or with a schema registry)

